Sri Lanka Institute of Information Technology

Programming Applications and Frameworks (IT3030)

Final Assignment Report

GROUP ID: Y3S1-WE-05

Smart Campus Operations Hub

Student ID	Student Name
IT23580930	Rathnayake R.M.L.U
IT23561670	Wijesinghe W.H.N
IT23585744	Jayasinghe R.M.S.K
IT23569522	Medhani W.A.P.

Date: 28.04.2026

Table of Contents

1. Introduction 3
2. Functional Requirements & Non-Functional Requirements 4
3. Overall Architecture Diagram 9
4. REST API Architecture diagram 10
5. Frontend Architecture Diagram 11
6. System Functions 12
7. GitHub 41
8. References 42

1. Introduction

The **Smart Campus Operations Hub** is a full-stack, production-inspired web application designed to modernize and centralize university operational processes into a single digital platform. Universities typically manage multiple independent systems for authentication, facility bookings, academic data, and incident handling, leading to inefficiencies, poor user experience, and limited visibility across operations. This project addresses these challenges by integrating all core services into a unified, scalable system.

The platform is implemented as a **monorepo architecture**, consisting of a **Spring Boot (Java 21) RESTful backend** and a **React + Vite frontend application**.The system follows modern software engineering best practices, including layered backend architecture, role-based frontend design, and token-based authentication with session and two-factor authentication (2FA) support.

At its core, the system is designed around **role-driven workflows**, enabling different types of users—such as students, technicians, administrators, and academic staff—to interact with the system based on their responsibilities. End users can manage bookings, submit incident tickets, and receive notifications, while technicians handle ticket resolutions, and administrators oversee system operations, approvals, and academic configurations.

The application implements several key functional domains required by the assignment scenario:

- **Authentication and User Onboarding:** Secure login, registration, OAuth integration, and 2FA-based account protection
- **Incident Ticketing System:** Creation, tracking, and resolution of maintenance issues with attachments and workflow states
- **Resource and Booking Management:** Management of university facilities with availability validation and approval workflows
- **Academic Data Management:** Handling faculties, programs, modules, and user registrations
- **Notification System:** Real-time updates for bookings, tickets, and system events

A **hybrid persistence model** is used to optimize data handling, combining **MongoDB** for flexible document-based storage (authentication, sessions, operational data) and **relational databases (JPA-based)** for structured academic and transactional data. Additionally, file uploads such as ticket evidence are managed through a dedicated storage mechanism.

The system is developed to meet key **non-functional requirements**, including scalability, maintainability, security, and usability. It follows RESTful principles, ensures proper API design with meaningful endpoints and status codes, and provides a responsive and user-friendly interface.

Overall, the Smart Campus Operations Hub demonstrates the design and implementation of a **modern, enterprise-style web application**, fulfilling the assignment requirements while extending functionality through additional features such as advanced security handling, role-based workflows, and comprehensive operational management.

2. Functional Requirements & Non-Functional Requirements

Functional Requirements

The functional requirements define the core features and system behaviors that the Smart Campus Operations Hub must provide.

1. User Authentication & Onboarding

I implemented a secure and user-friendly authentication system that supports multiple login methods.

Features Completed:

- User registration using email and password.
- Secure login using email/password credentials.
- Google OAuth Sign-In integration for faster access.
- First-time user onboarding flow after registration/login.
- Password configuration for new users.
- Session handling and secure token-based authentication.
- Automatic redirection to the correct dashboard after login.

Supported Roles:

- USER
- ADMIN
- TECHNICIAN

2. Two-Factor Authentication (2FA)

To improve account security, I implemented optional two-factor authentication.

Features Completed:

- Users can enable 2FA during first login setup.
- Users can skip setup initially and enable it later through Settings.
- Two verification methods supported:

Method 1 – Email Verification Code

- OTP code sent to registered email.

Method 2 – Authenticator App (TOTP)

- QR code scanning support.
- Compatible with Google Authenticator and similar apps.

Additional Features:

- Verification screen during login.
- Secure code validation before dashboard access.

3. Role-Based Access Control

I implemented access control logic to ensure users only access features relevant to their role.

Features Completed:

USER Access:

- Book resources
- View bookings
- Submit tickets
- View notifications

ADMIN Access:

- Admin Console
- Booking approvals/rejections
- Resource management
- Technician management
- Targeted notifications
- System settings

TECHNICIAN Access:

- Assigned tickets
- Accept/reject assignments
- Resolve tickets
- View withdrawn tickets
- Technician notifications

3. Resource Management (Facilities & Assets)

- Admin users must be able to create, update, and delete resources (e.g., lecture halls, labs, equipment).
- Each resource must include attributes such as type, capacity, location, availability, and status.
- Users must be able to search and filter resources based on different criteria.

4. Booking Management

- Users must be able to request bookings for available resources.
- The system must validate booking requests to prevent time conflicts.
- Booking workflow must support states: **PENDING → APPROVED / REJECTED → CANCELLED**.
- Admin users must be able to approve or reject bookings with a reason.
- Users must be able to view and manage their bookings.

5. Incident Ticketing System

- Users must be able to create tickets with title, category, description, priority, and contact details.
- Users must be able to upload attachments (e.g., images as evidence).
- Tickets must follow a lifecycle: **OPEN → IN_PROGRESS → RESOLVED → CLOSED / REJECTED**.
- Technicians must be able to accept, update, and resolve tickets.
- Users and staff must be able to add comments and updates to tickets.
-

6. Notification System

I developed the complete notification module to improve communication between users, admins, and technicians.

In-App Notifications:

- Notification bell icon
- Unread count badge
- Notification dropdown preview
- Mark as read
- Mark all as read
- Full notifications page

Email Notifications:

- Email alerts sent to users when enabled.
- SMTP integration support.

User Settings:

Users can enable/disable:

- Email notifications
- In-app notifications

7. Targeted Notifications (Admin Feature)

I developed an admin communication panel for sending targeted notifications.

Features Completed:

Admin can send notifications to:

- USER role
- ADMIN role
- TECHNICIAN role
- Everyone

Delivery Channels:

- In-App Web Notification
- Email Notification
- Both

Additional Options:

- Subject
- Message
- Priority
- Schedule time

8. Workflow Notifications Implemented

Booking Module:

- Booking request submitted
- Booking approved
- Booking rejected

Ticket Module:

- New ticket created

- Technician assigned
- Technician accepted assignment
- Technician rejected assignment
- Ticket resolved
- Ticket withdrawn
- Ticket comments / updates

Smart Messaging:

Notifications include:

- Ticket/booking reference number
- Resource name
- Admin / Technician actor name
- Status details

9. Academic Data Management

- Admin users must manage academic structures such as faculties, programs, modules, and intakes.
- The system must support registration of students, lecturers, and staff roles.
- Data must be organized and retrievable efficiently.

10. File Handling

- The system must support uploading and retrieving files (e.g., ticket attachments).
- File uploads must be validated and securely stored.

Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system.

1. Performance

- The system should handle multiple concurrent users efficiently.
- API responses should be optimized for low latency.
- Database queries must be optimized for performance.

2. Scalability

- The system should support scaling to accommodate increasing users and data.
- The architecture should allow future extension of modules (e.g., additional services).

3. Security

- All sensitive data must be protected using encryption and secure protocols.
- Authentication must use secure token-based mechanisms (Bearer tokens).
- Role-based authorization must be strictly enforced.
- File uploads must be validated to prevent malicious content.
- Support for two-factor authentication (2FA) enhances account security.

4. Usability

- The user interface must be intuitive and easy to navigate.
- The system should provide clear feedback messages (e.g., success, error states).
- Role-based dashboards should improve user experience.

5. Maintainability

- The codebase must follow clean architecture principles (layered backend, modular frontend).
- Proper naming conventions and documentation must be maintained.
- The system should be easy to update and extend.

6. Reliability

- The system should ensure consistent operation with minimal downtime.
- Error handling must be implemented to prevent crashes and data loss.
- Backup and recovery mechanisms should be considered.

7. Compatibility

- The frontend must be accessible through modern web browsers.
- The system should support different screen sizes (responsive design).

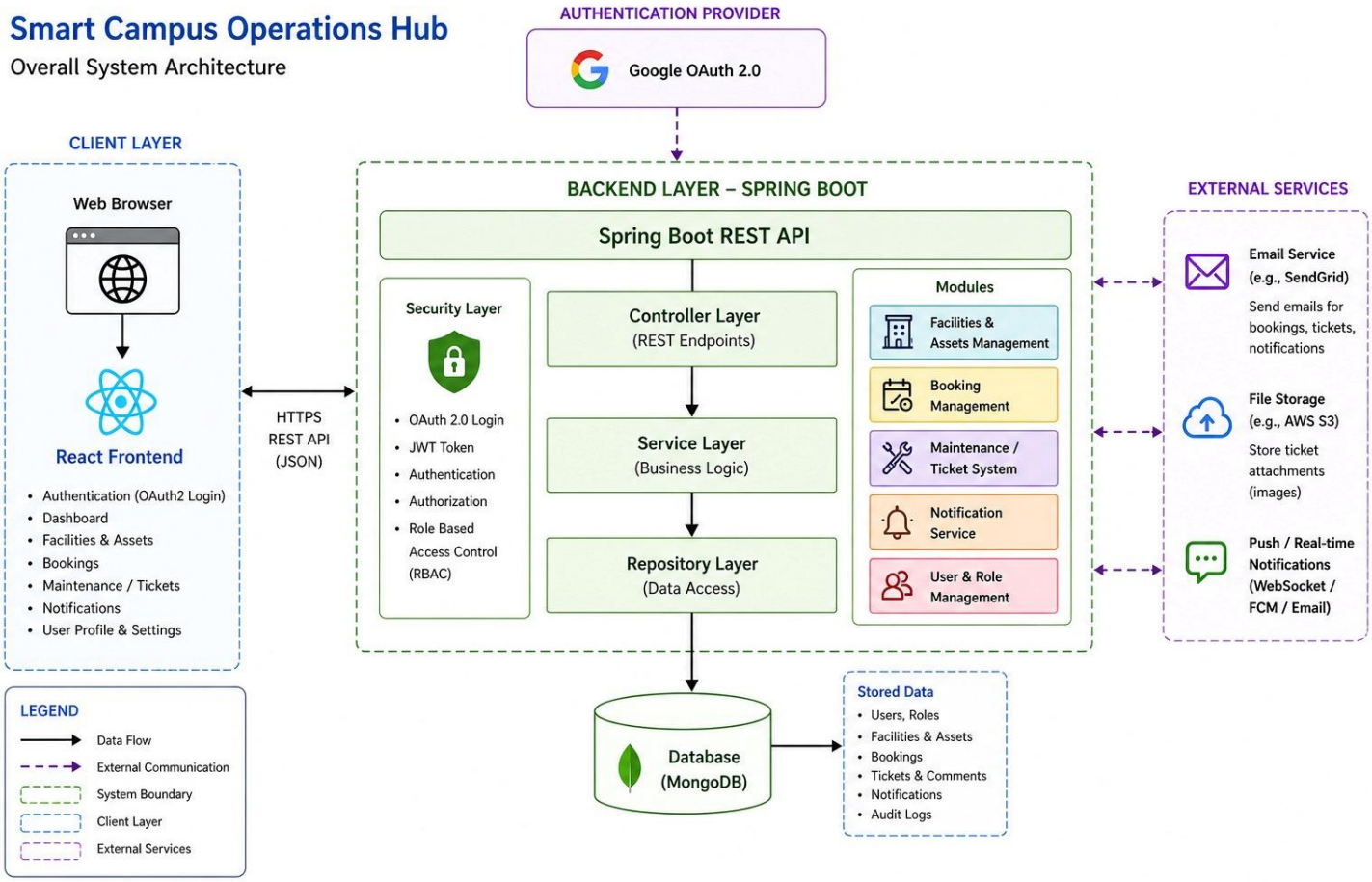
8. Testability

- The system should support unit, integration, and end-to-end testing.
- Testing tools such as Playwright must validate key workflows.

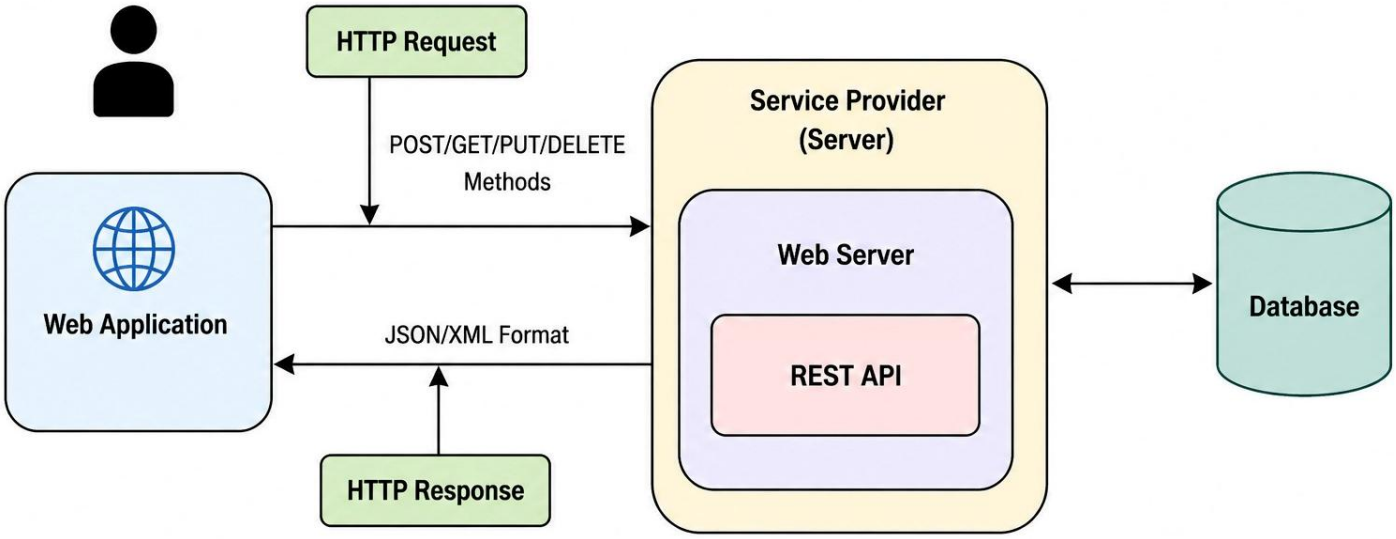
9. Data Integrity

- The system must ensure accurate and consistent data across MongoDB and relational databases.
- Transactions must be handled properly to avoid data inconsistencies.

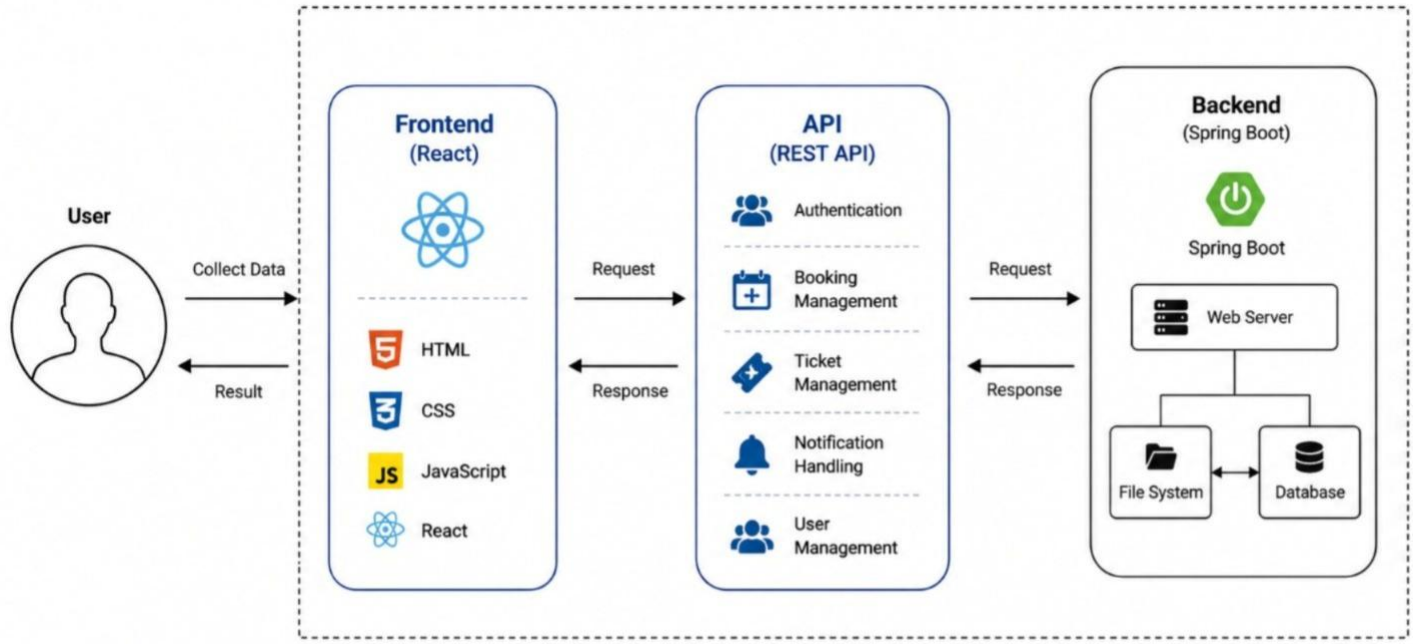
3. Overall Architecture Diagram



4. REST API Architecture diagram



5. Frontend Architecture Diagram



6. System Functions

I. IT23580930 – (Member 4: Notifications + role management + OAuth integration improvements)

Frontend Contributions (React)

My primary responsibility in the Smart Campus Operations Hub project was developing the **Authentication**, **Role-Based Access Control**, and **Notification** features on the frontend. My main focus was to improve system security, simplify user login methods, and enhance communication between users, administrators, and technicians through a modern and user-friendly interface.

1.1 Google Sign-In & Two-Factor Authentication UI

Components / Files:

- frontend/src/context/AuthProvider.jsx
- frontend/src/components/LoginPanel.jsx
- frontend/src/components/GoogleIdentityButton.jsx
- frontend/src/pages/SignupPage.jsx
- frontend/src/routes/ProtectedRoute.jsx
- frontend/src/services/authService.js

Key Contributions:

- Implemented **Google Sign-In** to provide a faster and more convenient login experience.
- Integrated frontend authentication screens with backend Google OAuth verification APIs.
- Added **Two-Factor Authentication (2FA)** setup during the first-time login process.
- Allowed users to choose their preferred verification method:
  - Google Authenticator App
  - Email Verification Code (OTP)
- Added a **Skip** option during first-time setup for better usability.
- Allowed users who skipped setup to enable 2FA later through the Security Settings page.
- Improved the login flow while maintaining both usability and security.
- Ensured users are redirected to the correct dashboard according to their assigned role (USER, ADMIN, TECHNICIAN).
- Improved protected route handling to prevent unauthorized access to restricted pages.

1.2 Notification User Interface

Components / Files:

- frontend/src/components/notifications/NotificationBell.jsx
- frontend/src/pages/NotificationsPage.jsx
- frontend/src/pages/admin/AdminNotificationsPage.jsx
- frontend/src/services/notificationApi.js

- frontend/src/services/notificationAudienceService.js
- frontend/src/services/portalDataService.js

Key Contributions:

- Developed the notification bell icon with an **unread notification count badge**.
- Added a dropdown panel to preview recent notifications quickly.
- Implemented **mark-as-read** functionality when users open notifications.
- Added navigation improvements for both user and admin notification pages.
- Implemented role-based targeted notification sending through the admin interface.
- Enabled administrators to send notifications directly to selected user roles:
  - USER
  - ADMIN
  - TECHNICIAN
- Added email notification support through the frontend UI.
- Allowed users to enable or disable email notifications through system settings.
- Improved the overall notification experience with a cleaner and more responsive interface.

Backend Contributions (Spring Boot REST API)

In addition to frontend development, I also contributed significantly to backend development related to secure authentication, notifications, and user communication features.

2.1 Authentication & Security

Key Contributions:

- Implemented backend verification for Google Sign-In authentication.
- Added support for **Two-Factor Authentication (2FA)**.
- Supported two verification methods:
  - Authenticator App verification codes
  - Email OTP verification
- Developed an optional first-time security setup process.
- Allowed users to skip 2FA setup initially and enable it later through account settings.
- Improved account security without negatively affecting user convenience.
- Contributed to role-based access restrictions using secure backend authorization logic.

2.2 Authentication & User Management APIs

Worked on APIs such as:

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/google/signup-session
- POST /api/v1/auth/google
- POST /api/v1/auth/dev-login
- POST /api/v1/auth/verify-2fa
- POST /api/v1/auth/resend-email-otp

- POST /api/v1/auth/first-login/change-password
- POST /api/v1/auth/first-login/select-2fa-method
- GET /api/v1/auth/me
- GET /api/v1/auth/signup-requests/{requestId}/status
- GET /api/v1/auth/signup-requests/{requestId}/activate
- POST /api/v1/auth/signup-requests/{requestId}/activate
- GET /api/v1/account/security-settings
- PUT /api/v1/account/security-settings
- POST /api/v1/account/security-settings/authenticator/start
- POST /api/v1/account/security-settings/authenticator/verify
- POST /api/v1/account/security-settings/authenticator/reset
- POST /api/v1/account/google-2fa-prompt/dismiss
- GET /api/v1/admin/signup-requests
- POST /api/v1/admin/signup-requests/{requestId}/approve
- POST /api/v1/admin/signup-requests/{requestId}/reject

Key Contributions:

- Strengthened user onboarding and secure account activation workflows.
- Improved role-aware authentication flows.
- Supported secure account setup, verification, and password initialization.

2.3 Notification APIs

Worked on APIs such as:

- GET /api/v1/notifications
- GET /api/v1/notifications/unread-count
- PATCH /api/v1/notifications/{id}/read
- PATCH /api/v1/notifications/read-all
- DELETE /api/v1/notifications/{id}
- POST /api/v1/admin/notifications
- GET /api/v1/admin/notifications/history
- GET /api/v1/admin/notification-settings
- PUT /api/v1/admin/notification-settings
- POST /api/v1/notifications/audience
- POST /api/v1/notifications/email
- GET /api/v1/portal-data/notification-feed
- PUT /api/v1/portal-data/{key}

Key Contributions:

- Added unread notification count support for dashboard displays.
- Enabled recent notifications for dropdown previews.
- Implemented mark-as-read and mark-all-as-read functionality.

- Added targeted notifications based on user roles.
- Added email notification delivery for selected users.
- Allowed users to manage notification preferences through settings.
- Ensured notification records are securely stored in the database.
- Improved overall communication efficiency across the system.

2.4 Notification Workflow Integration

Key Contributions:

- Added notifications for booking approvals and rejections.
- Added ticket-related notifications such as:
    - Technician assigned
    - Assignment accepted
    - Assignment rejected
    - Ticket resolved
    - Ticket withdrawn
- Included meaningful notification messages with:
    - Booking or ticket reference numbers
    - Relevant action details
    - Admin / technician actor names
    - Status updates
- Improved real-time communication between users, administrators, and technicians.
- Helped create a more responsive and user-focused system experience.

2.5 Email Notification / SMTP Integration

Key Contributions:

- Configured Gmail SMTP for outgoing email notifications.
- Integrated backend email sending using secure mail configuration.
- Enabled email alerts for important actions such as:
    - Booking approvals/rejections
    - Ticket assignment updates
    - Security verification codes
- Supported user preference-based email notifications.
- Improved system communication beyond in-app notifications.

Summary of My Contribution

My main contribution focused on building a secure and modern authentication system together with a complete notification platform. I successfully implemented Google OAuth login, optional Two-Factor Authentication, role-based access control, targeted notifications, unread notification tracking, email alerts, and workflow-based notifications for bookings and tickets. These features significantly improved system security, usability, and communication across all user roles.

Google Sign-In & Two-Factor Authentication UI

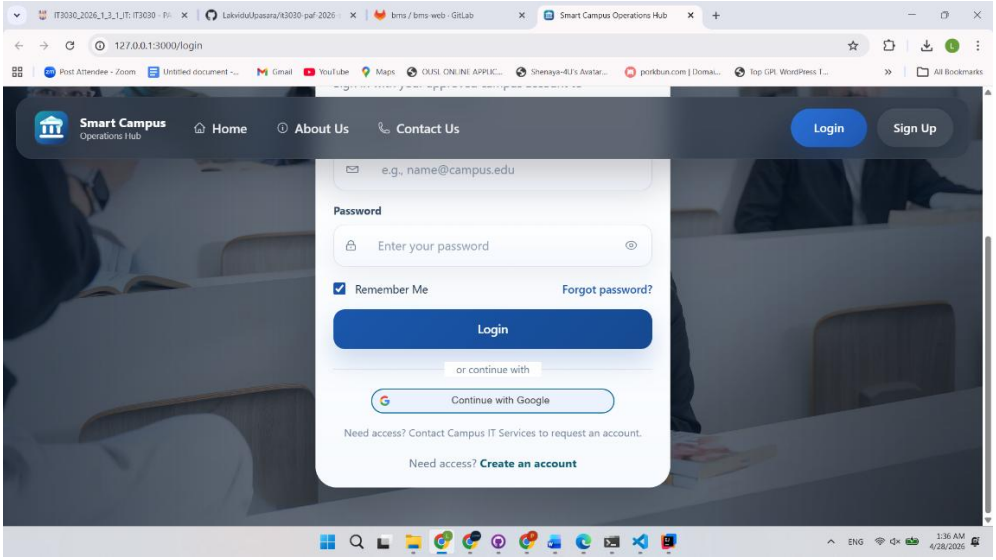


Figure 1.1: User Login Page with Google Authentication

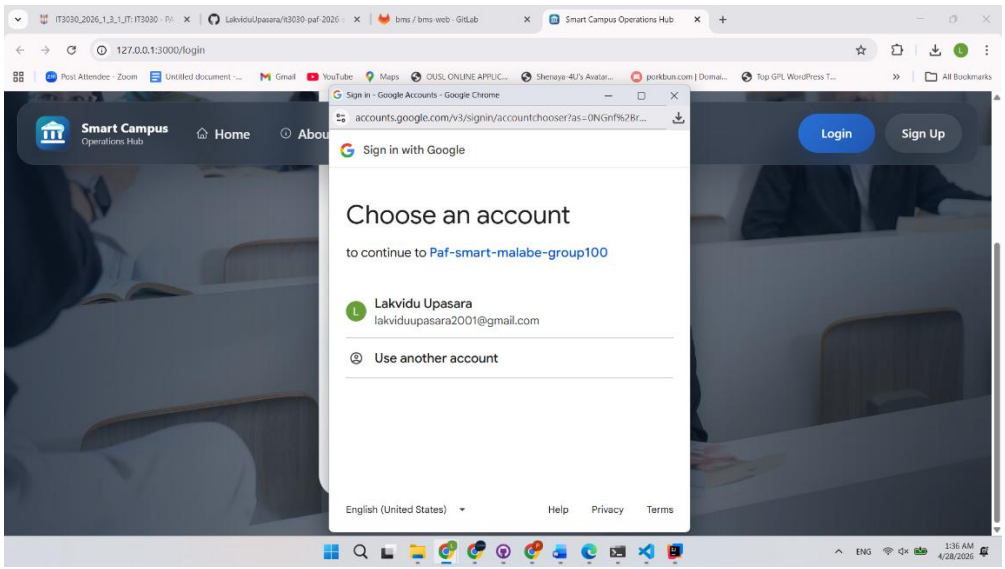


Figure 1.1a: User Login Page with Google Authentication

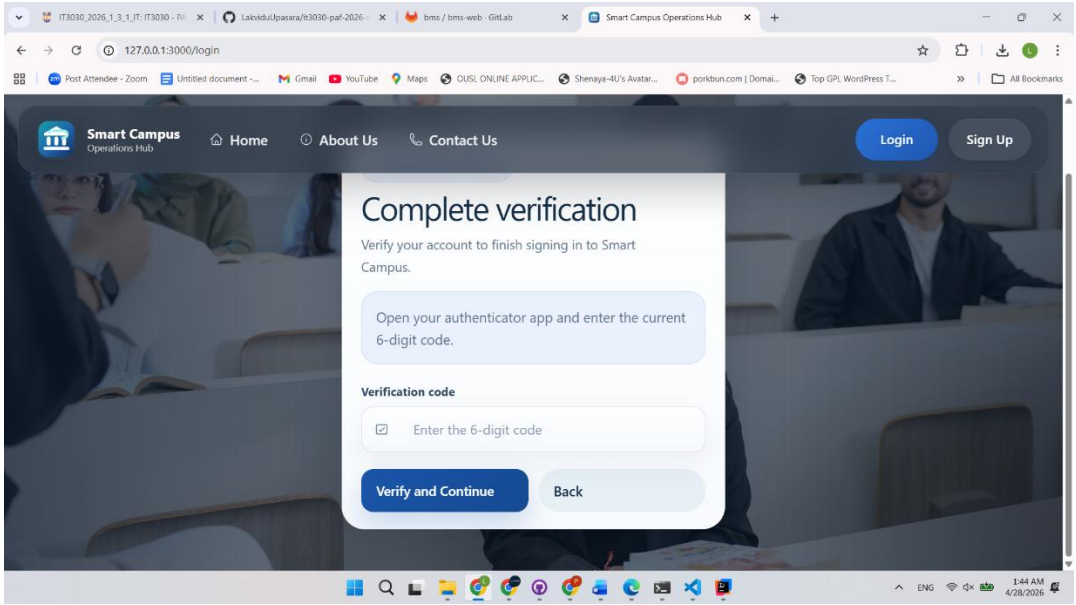


Figure 1.1c: User Login Two-Factor Authentication Verification Page

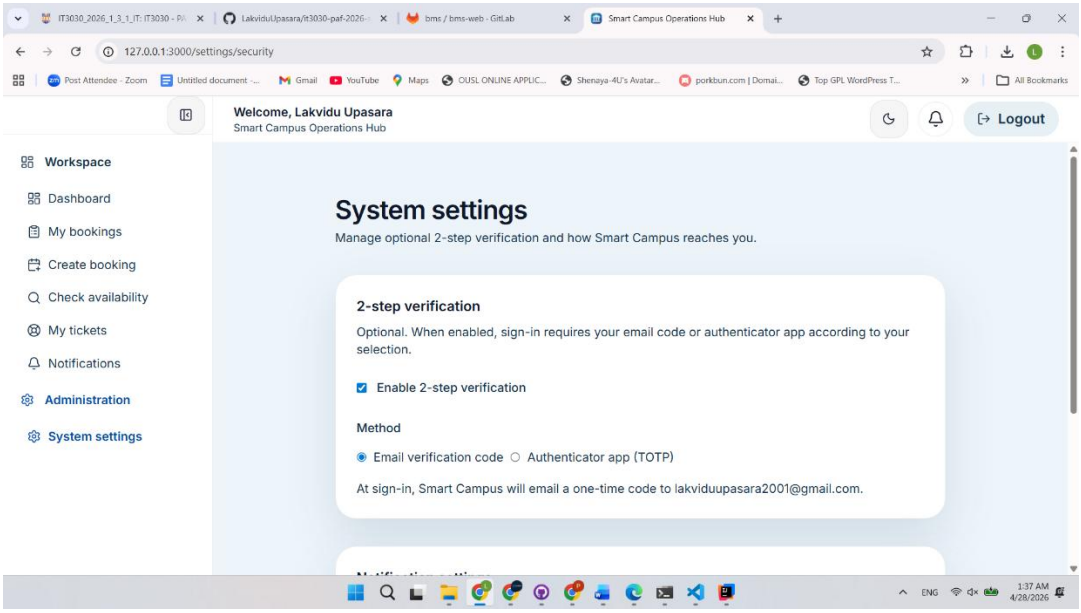


Figure 1.1d: System Settings Page for Enabling Two-Step Verification

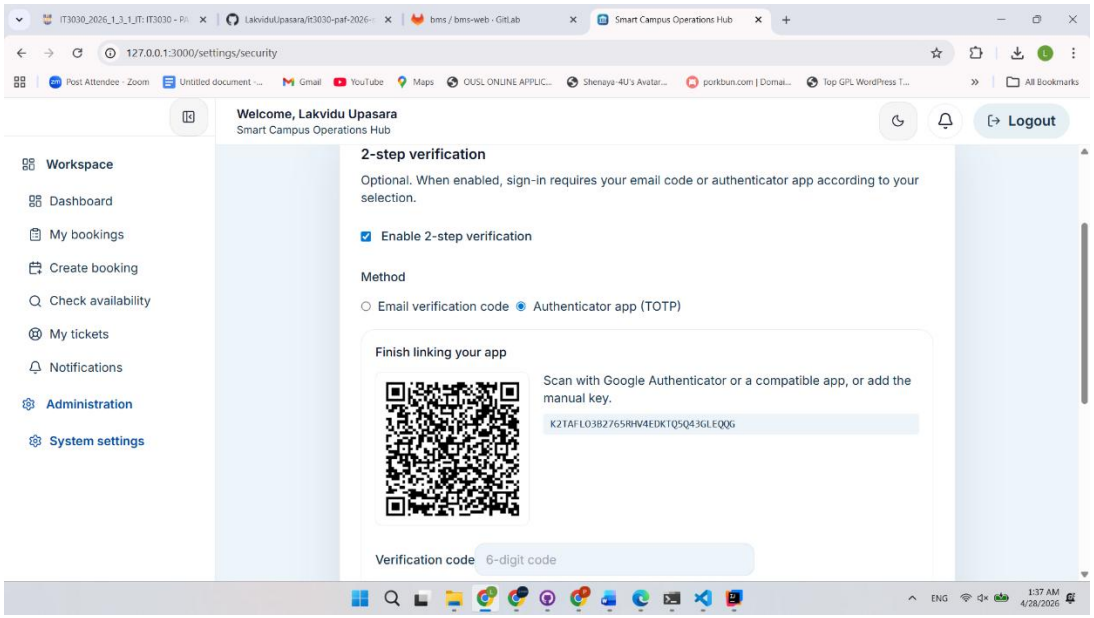


Figure 1.1E: Authenticator App (TOTP) Setup Interface with QR Code for Two-Factor Authentication

Notification User Interface

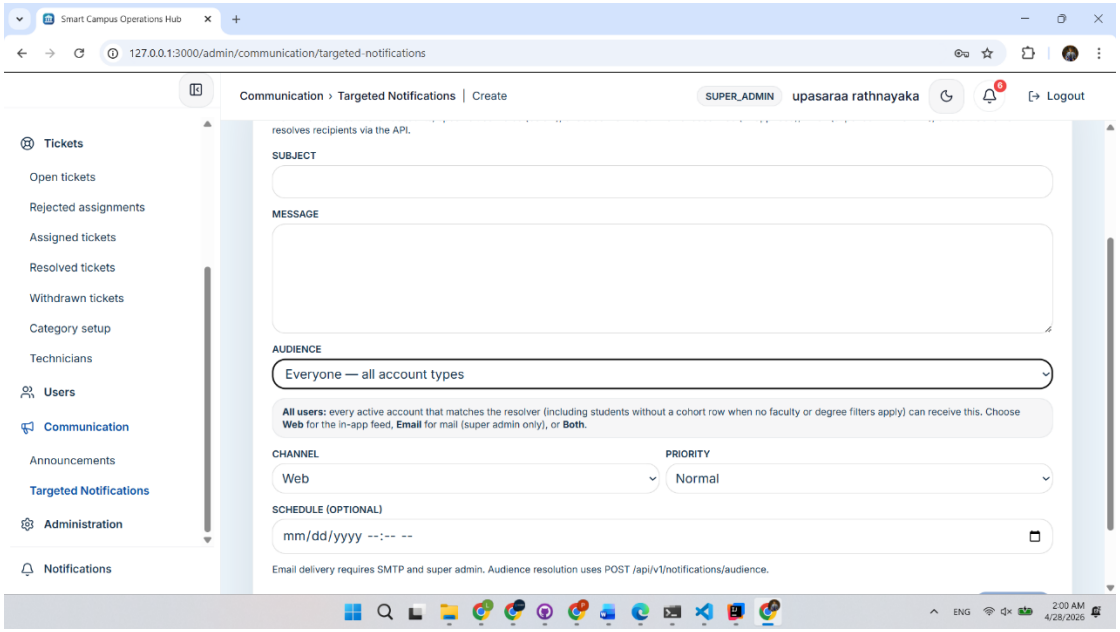


Figure 1.2a: Admin Targeted Notification Management Interface for Role-Based Communication

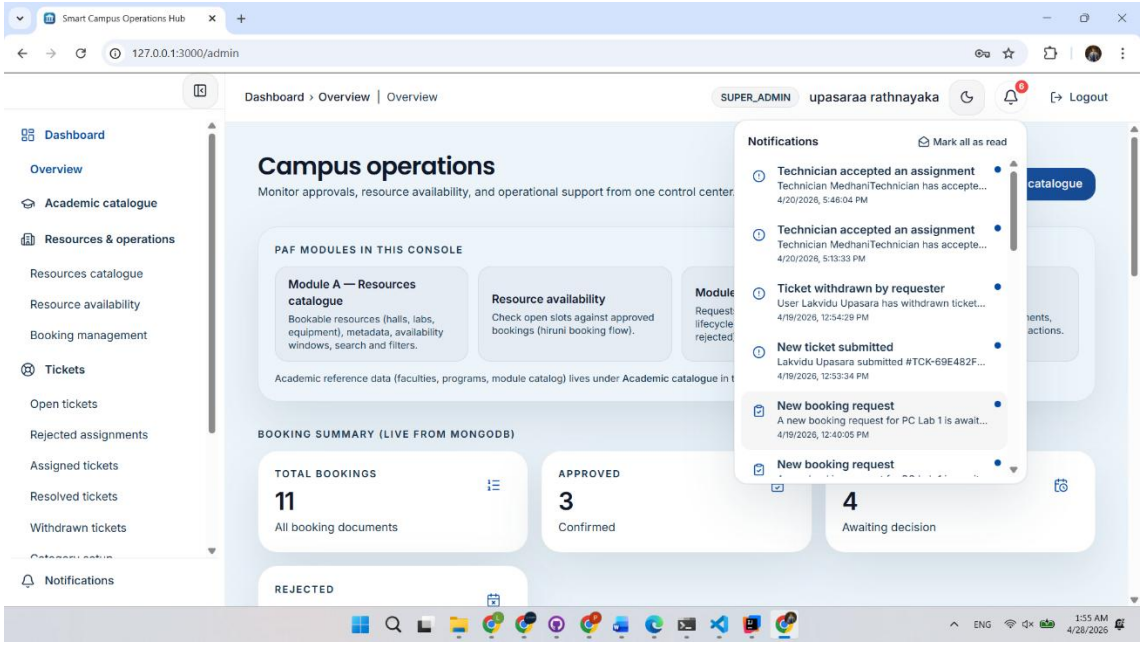


Figure 1.2b: Admin, User, and Technician Dashboard Notification Panel with Real-Time Booking, Ticket, and Targeted Notification Alerts

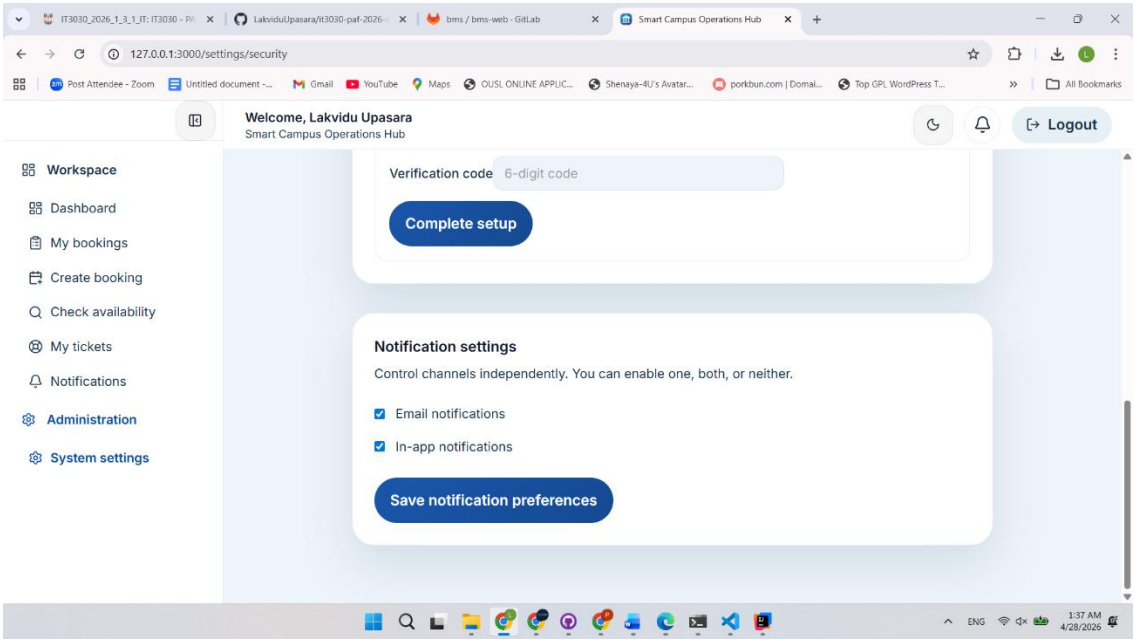


Figure 1.2c: Notification Preference Settings Interface for Email and In-App Alerts

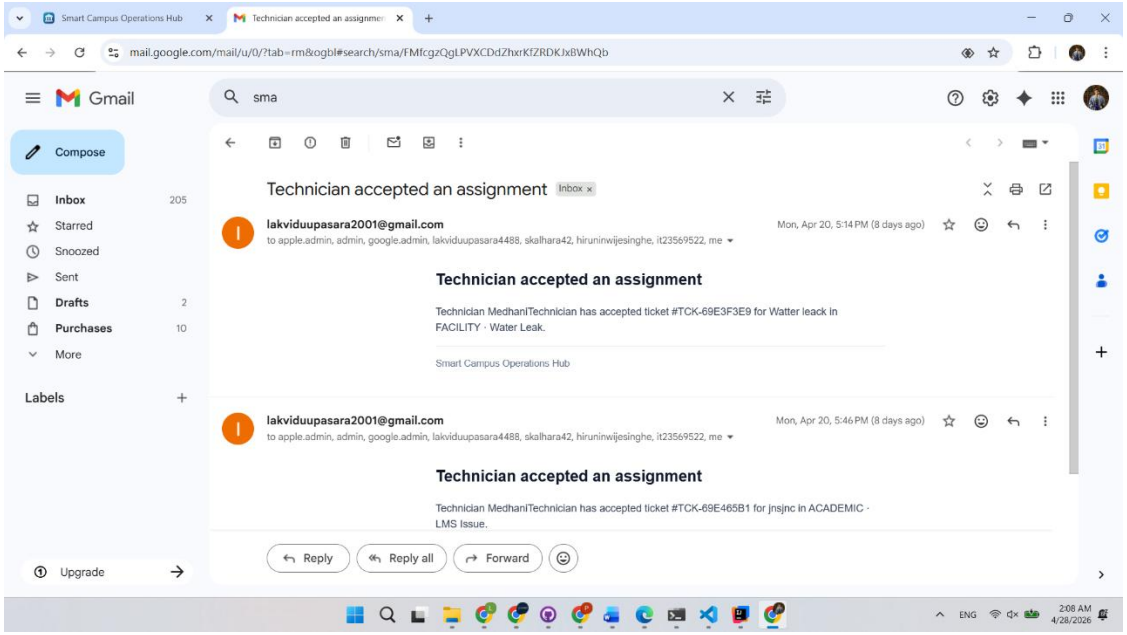


Figure 1.2e: Email Notification Delivery for Technician Assignment Acceptance Updates

II. IT23561670 - (Member 2: Booking workflow + conflict checking)

<u>Frontend Contributions (React)</u>

I was mainly involved in developing the **Booking Management functionality** on the frontend, focusing on building a smooth and user-friendly experience for handling bookings. This included enabling users to check resource availability, create booking requests, and manage their bookings efficiently, as well as supporting the administrative workflow for reviewing and approving requests.

Key Implementations

1.1 Booking Creation

Components:

- ResourceAvailabilityPage.jsx
- CreateBookingPage.jsx
- Developed interfaces for users to **view available resources** and create booking requests.
- Implemented a **structured booking form** with inputs such as date, time range, purpose.
- Added **validation mechanisms** to ensure correct and complete user input.
- Enabled **availability checking**, preventing users from selecting already booked time slots.
- Improved usability by providing **clear feedback for invalid or conflicting selections**.

1.2 Booking Management

Component:

- MyBookingsPage.jsx
- Designed a personalized dashboard to **view and manage user bookings**.
- Displayed booking details with clear status indicators:
    - PENDING
    - APPROVED
    - REJECTED
    - CANCELLED
- Implemented **booking cancellation functionality** with confirmation handling.
- Ensured **dynamic data rendering** by integrating with backend APIs.

1.3 Admin Booking Approval

Component:

- ApproveBookingsPage.jsx
- Developed an interface for administrators to **review and manage booking requests**.
- Implemented **approval and rejection workflow**, including reason-based rejection.
- Enabled efficient handling of booking requests through a **centralized booking queue**.

1.3 API Integration

- Integrated frontend components with backend .
- Managed:
    - API communication
    - Error handling
    - Loading states
- Ensured seamless **frontend–backend interaction**.

<u>Backend Contributions (Spring Boot REST API)</u>

I contributed to the backend implementation of the **Booking Management system**, focusing on business logic, validation, and workflow management.

Key Implementations

Booking Data Handling

- Implemented backend logic to **process and store booking requests**.
- Enabled retrieval of booking data via:

  GET /api/v1/bookings

  POST /api/v1/bookings

Boking Workflow Management

- Implemented booking lifecycle:

  PENDING → APPROVED / REJECTED → CANCELLED
- Developed logic for:
  - Booking creation
  - Approval/rejection handling
  - Cancellation processing

Conflict Detection

- Implemented validation logic to **prevent overlapping bookings** for the same resource and time slot.
- Ensured **data consistency and reliability** of booking operations.

Database Design (MongoDB)

- Designed booking-related data structures including:
  - Resource details
  - Booking time range
  - Status tracking
  - Approval/rejection information

Security and Validation

- Ensured only authenticated users can create bookings.
- Restricted approval/rejection actions to admin users.
- Validated all booking inputs before processing.

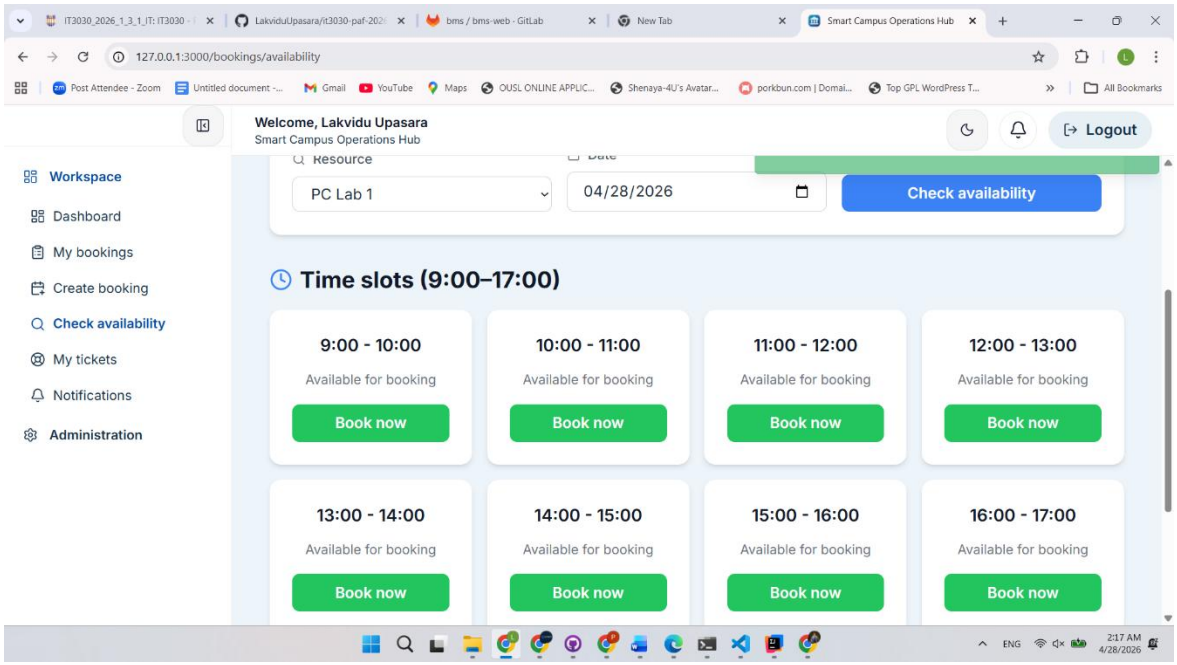


Figure 2.1a: Resource Availability Dashboard with Real-Time Time Slot Booking Status

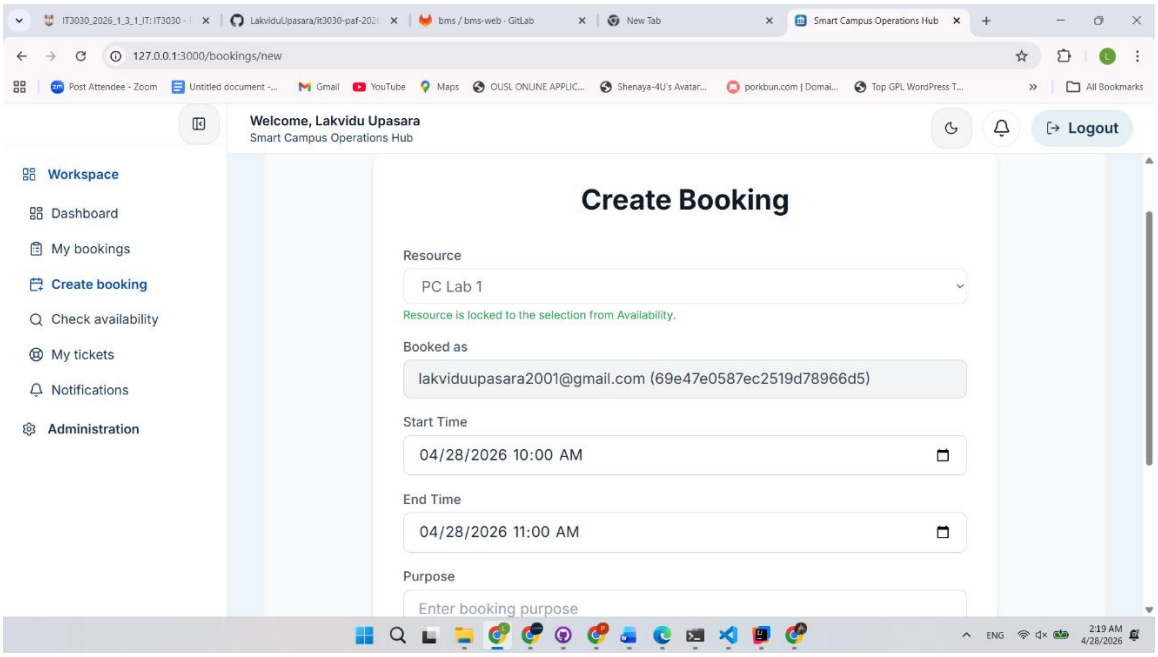


Figure 2.1b: Booking Creation Interface with Resource Selection and Scheduled Time Slot Reservation

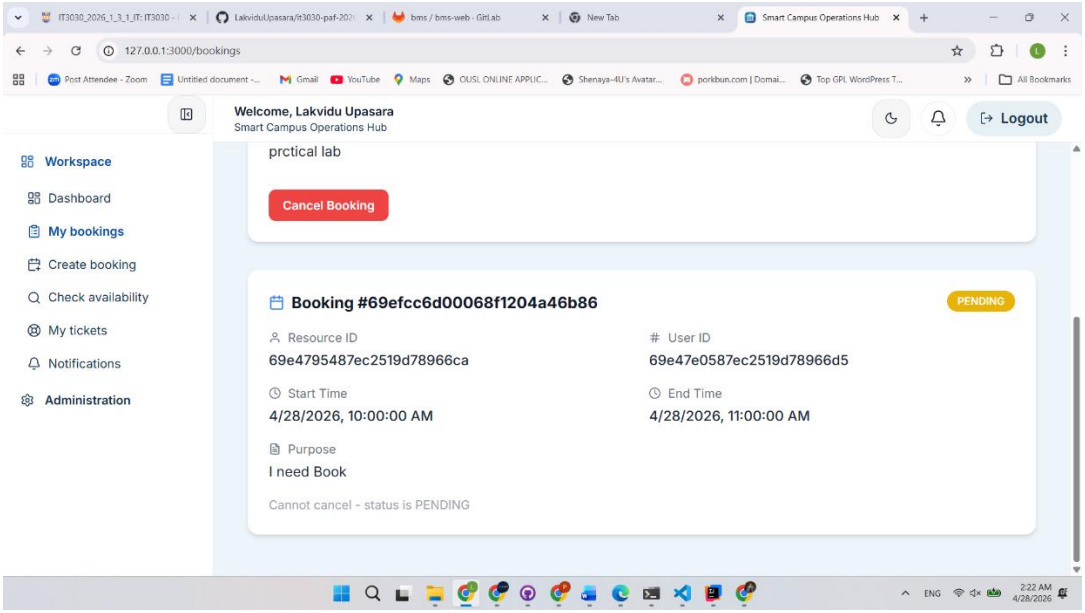


Figure 2.1c: User Booking Management Dashboard Showing Booking Status and Reservation Details

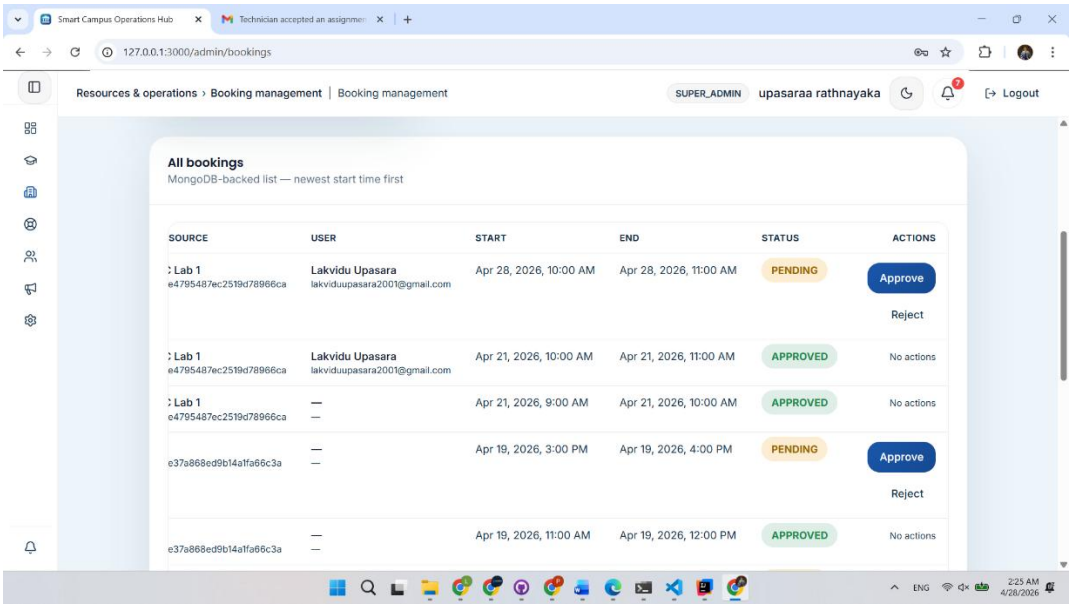


Figure 2.1e: Admin Booking Management Dashboard for Approving and Rejecting Reservation Requests

III. IT23585744 - (Member 1: Facilities catalogue + resource management endpoints)

Frontend Contributions (React)

I was mainly responsible for the facilities catalogue and faculty management interfaces, focusing on building admin-facing tools that support both operational resource handling and academic master-data management.

1.4 Facilities Catalogue and Resource Availability

Components:

- `ManageResourcesPage.jsx`
- `ResourceAvailabilityPage.jsx`
- `resourceService.js`

Key Implementations:

- Developed an admin resource catalogue to browse, filter, create, update, and delete campus resources.
- Added filter support by resource type, location, and minimum capacity to improve catalogue searchability.
- Implemented resource status handling and weekly availability window configuration for bookable resources.
- Built a resource availability page that checks selected time ranges against backend rules and approved bookings.
- Connected resource selection from the availability view into the booking flow to speed up booking creation.
- Integrated the facilities module into the admin operations workspace and navigation structure.

1.5 Faculty Management

Components:

- FacultiesAdminPage.jsx
- facultyService.js

Key Implementations:

- Developed a faculty administration interface with separate list and form modes for CRUD operations.
- Added search by faculty code or name together with status-based filtering.
- Implemented faculty creation with immutable faculty codes after initial creation.
- Added edit and soft-delete actions with confirmation handling and user feedback.

- Integrated faculty master data so it can be reused across academic and registration workflows.

Backend Contributions (Spring Boot REST API)

I contributed to the backend implementation for both resource management and faculty management, focusing on REST API design, validation, role-aware access control, and MongoDB persistence.

1.6 Resource Management Endpoints and Business Logic

Key Implementations:

- Designed and implemented resource REST endpoints under /api/v1/resources.
- Created a MongoDB-backed Resource model including name, type, capacity, location, status, and embedded availability windows.
- Implemented filtering logic for type, status, minimum capacity, and partial location matching.
- Added validation for availability windows, including required values, correct start/end ordering, duplicate prevention, same-day overlap prevention, and anchorDate to dayOfWeek consistency checks.
- Implemented /api/v1/resources/{id}/availability so availability checks follow the same rules as booking validation.
- Restricted resource create, update, and delete operations to authorized operational roles while allowing catalogue reads for signed-in users.

1.7 Faculty Management Endpoints and Data Handling

Key Implementations:

- Designed and implemented faculty REST endpoints under /api/v1/faculties.
- Created a MongoDB-backed faculty document model with code, name, status, timestamps, and soft-delete support.
- Added faculty code normalization and validation to enforce clean academic master data.
- Prevented duplicate faculty codes and validated required faculty names before saving data.
- Implemented soft-delete behaviour instead of hard deletion for safer academic data maintenance.
- Refreshed academic reference caches after faculty create, update, and delete operations so dependent academic modules remain synchronized.
- Supported public faculty reads for signup and registration dropdowns while keeping mutating endpoints restricted to authorized roles.

Cross-Module Impact

- The resource catalogue directly supports booking creation, booking conflict checks, and resource availability validation.

- The faculty catalogue acts as foundational academic reference data reused by degree-program, intake, module, and registration workflows.

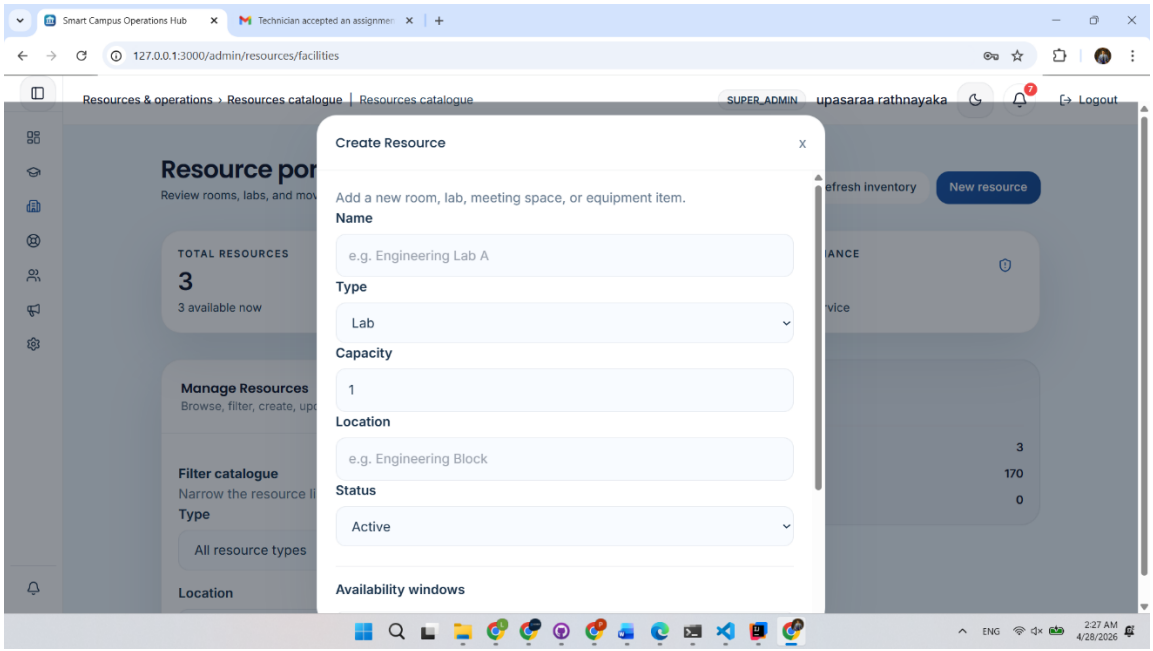


Figure 3.1: Admin Resource Management Interface for Creating New Campus Resources

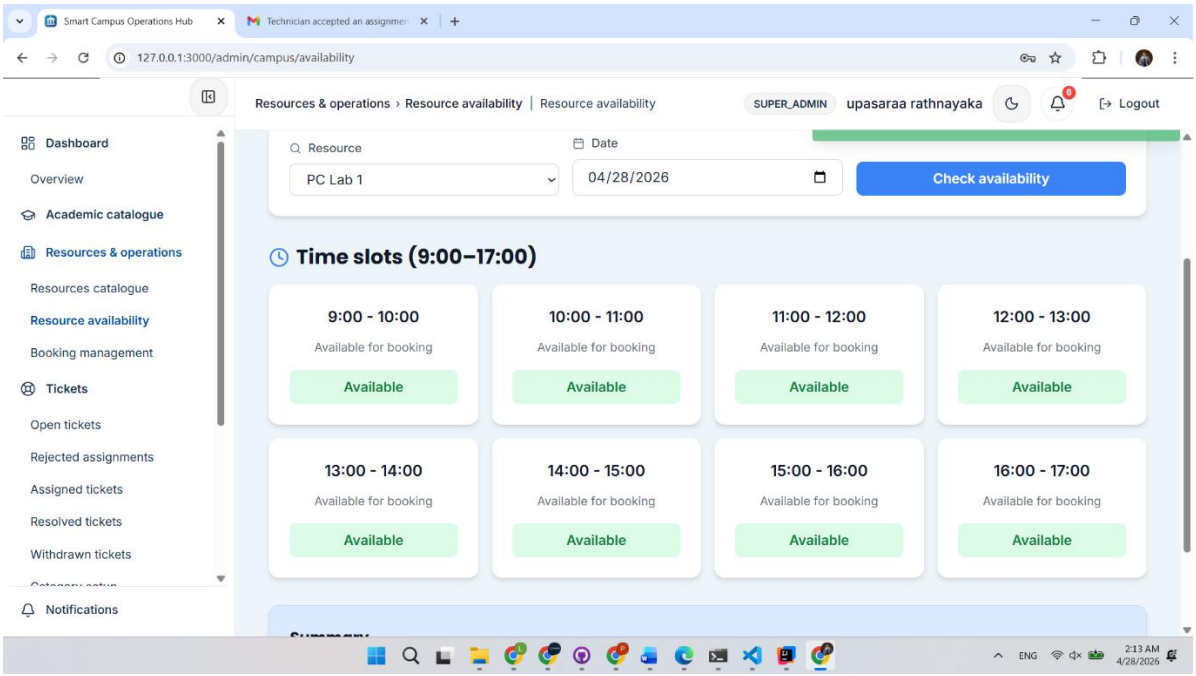


Figure 3.2: Admin Dashboard Resource Availability Interface with Real-Time Booking Time Slot Status

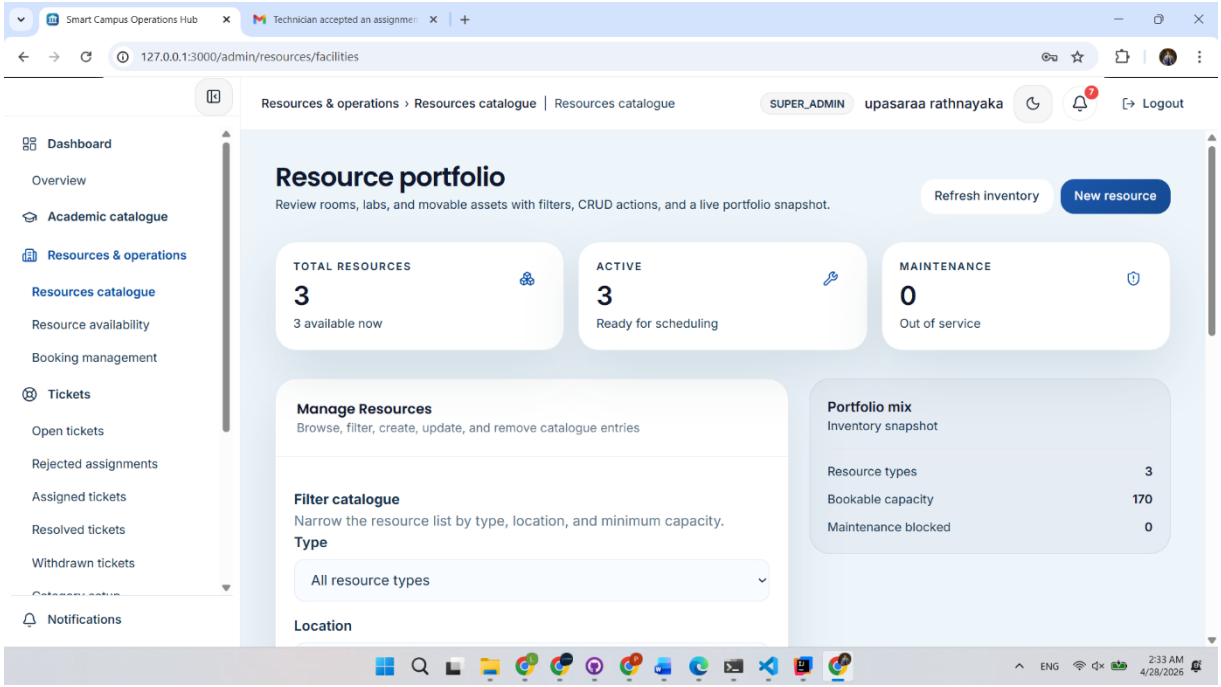


Figure 3.3a: Admin Dashboard Resource Portfolio Interface for Managing Campus Facilities and Assets

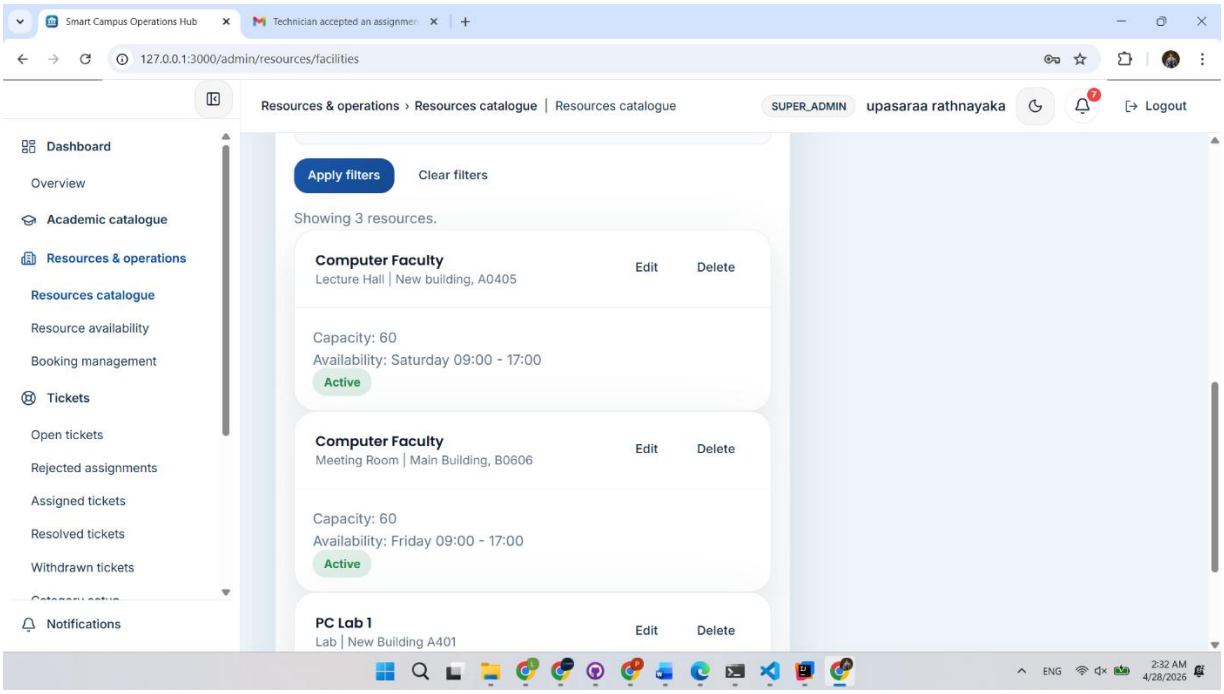


Figure 3.3b: Admin Dashboard Resource Portfolio Interface for Managing Campus Facilities and Assets

IV. IT23569522 - (Member 3: Incident tickets + attachments + technician updates)

Frontend Contributions (React)

I was primarily responsible for designing and implementing the **user-facing and technician interfaces** for the Incident Ticketing System, ensuring a complete and intuitive workflow from ticket creation to resolution.

Key Implementations

User Portal – Incident Reporting & Tracking

Component: MyTicketsPage.jsx

I developed a fully functional interface for students and staff to create, manage, and track incident tickets.
Key Features:

- Smart Ticket Creation
  - Built dynamic forms using *react-hook-form* with *Zod validation*
  - Ensured input validation for required fields, formats, and constraints
- AI-Powered Suggestions
  - Integrated getTicketSuggestion service
  - Automatically suggests category and subcategory based on user input
- Evidence Upload System
  - Enabled image uploads (max 3 files)
  - Implemented:
    - Image preview generation
    - Removal of selected files before submission
- Metadata Encoding Mechanism
  - Designed a custom META_MARKER system to embed:
    - Location
    - Priority
    - Contact details
  - Ensured seamless backend compatibility without altering API structure
- Ticket History & Lifecycle View
  - Users can:
    - View submitted tickets
    - Track status changes

        - Withdraw tickets when needed
- PDF Export Feature
    - Implemented downloadable PDF reports for resolved tickets
    - Provides a formal record of issue resolution

Technician Portal – Task Management Dashboard

Component: TechnicianDashboardPage.jsx

I implemented a dedicated workspace for technicians to manage assigned incidents.
Key Features:

- Task Management Dashboard
    - View and filter assigned tickets
    - Categorize tickets into:
        - *In Progress*
        - *Resolved*
- Assignment Workflow
    - Accept or reject assigned tickets
    - Manage ticket state transitions
- Technician Notes System
    - Add, edit, and delete progress updates
    - Enables structured communication within tickets
- Technician Evidence Upload
    - Separate upload system for technician proof (images)
    - Clearly distinguishes technician input from user-submitted evidence

Admin Portal – Incident Control & Monitoring

Component: ManageTicketsPage.jsx

I developed an advanced admin interface for monitoring and controlling all tickets.
Key Features:

- Queue Management System
    - Categorized ticket views:
        - Open

        - Assigned
        - Resolved
        - Withdrawn
- Technician Assignment Module
    - Assign/reassign tickets via modal interface
    - Displays technician availability dynamically
- Comprehensive Ticket View
    - Displays full ticket lifecycle:
        - User evidence
        - Technician notes
        - Technician-uploaded evidence
    - Provides complete audit visibility

Backend Contributions (Spring Boot REST API)

I contributed to the design and implementation of the **Incident Ticketing REST API**, ensuring proper architecture, validation, and integration with the frontend.

Key Implementations

- **REST API Development**
    - Designed and implemented endpoints such as:
        - POST /api/v1/tickets → Create ticket
        - GET /api/v1/tickets/me → Retrieve user tickets
        - POST /api/v1/tickets/{id}/attachments → Upload evidence
        - PUT /api/v1/tickets/{id}/withdraw → Withdraw ticket

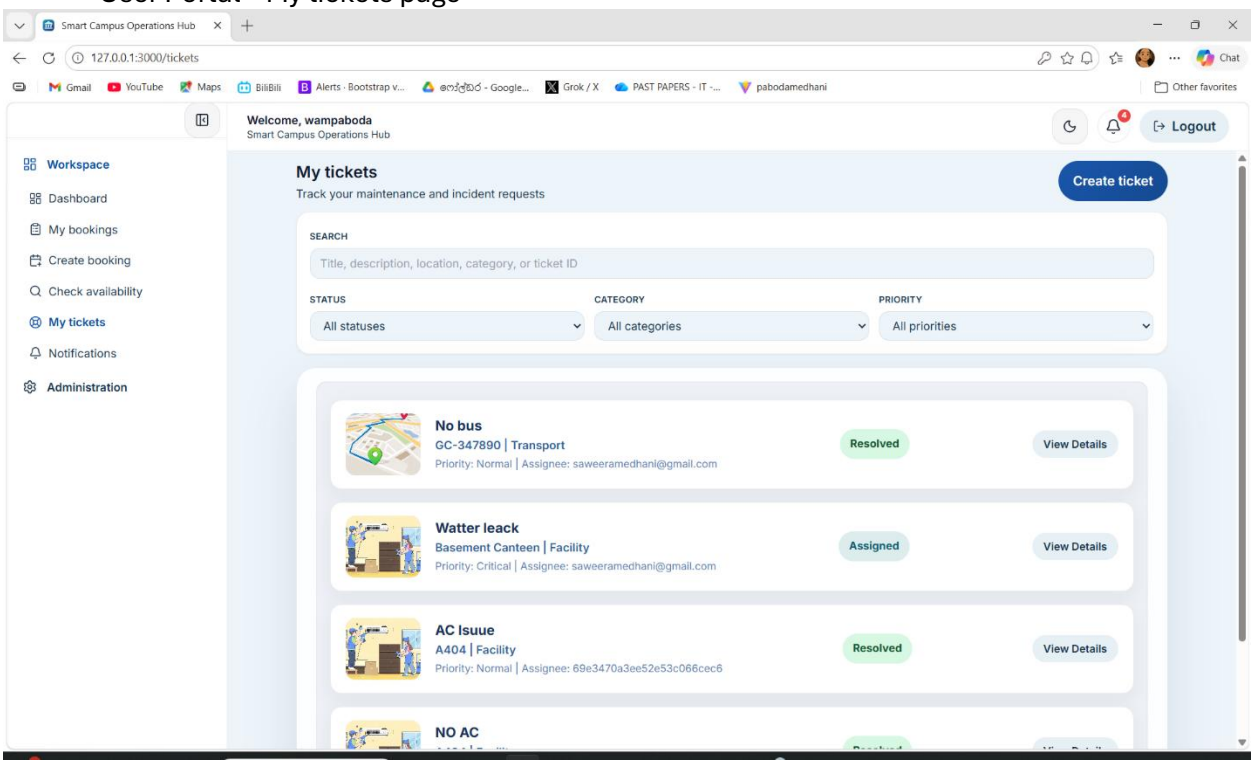
- **Business Logic (Service Layer)**
    - Implemented **ticket lifecycle workflow**:
        - OPEN → ASSIGNED → IN_PROGRESS → RESOLVED / WITHDRAWN
    - Managed:
        - Ticket updates
        - Withdrawal logic with reason tracking

        - Attachment handling

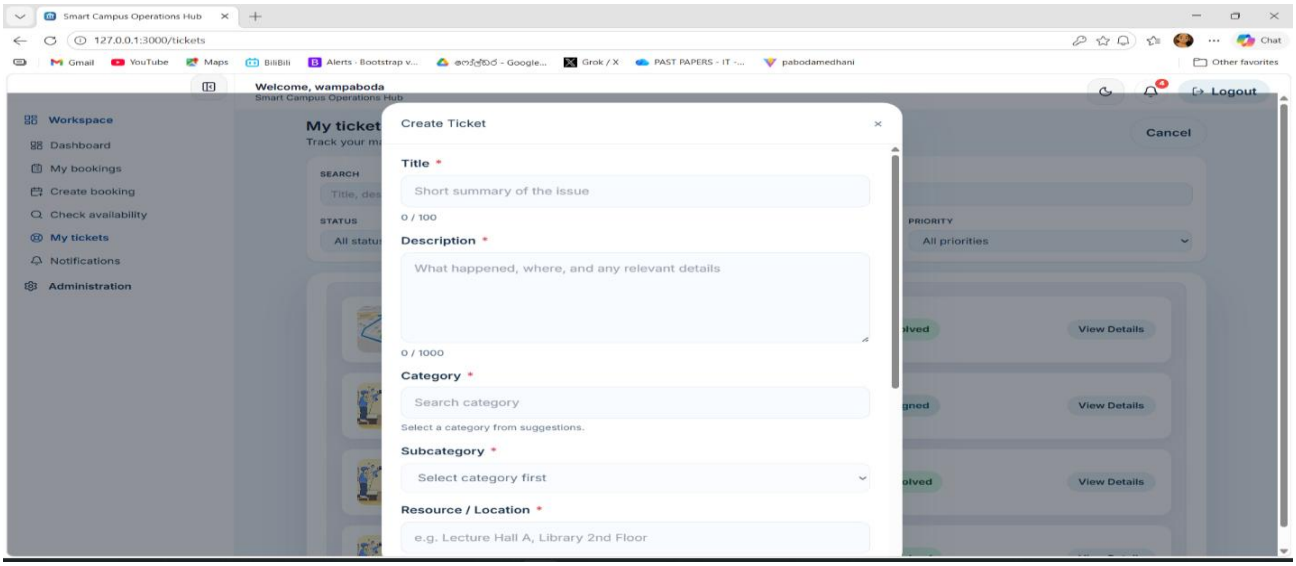
- **File Handling System**
    - Developed backend logic for:
        - Multipart file uploads
        - Secure storage and retrieval of ticket evidence
- **Data Processing**
    - Implemented parsing logic to extract metadata (e.g., location, priority) from ticket descriptions
- **Database Design (MongoDB)**
    - Designed **Ticket schema** including:
        - Title, description, category
        - Status and lifecycle tracking
        - Attachments
        - Technician updates and audit history

Screenshots Of UI – Incident & Ticketing System

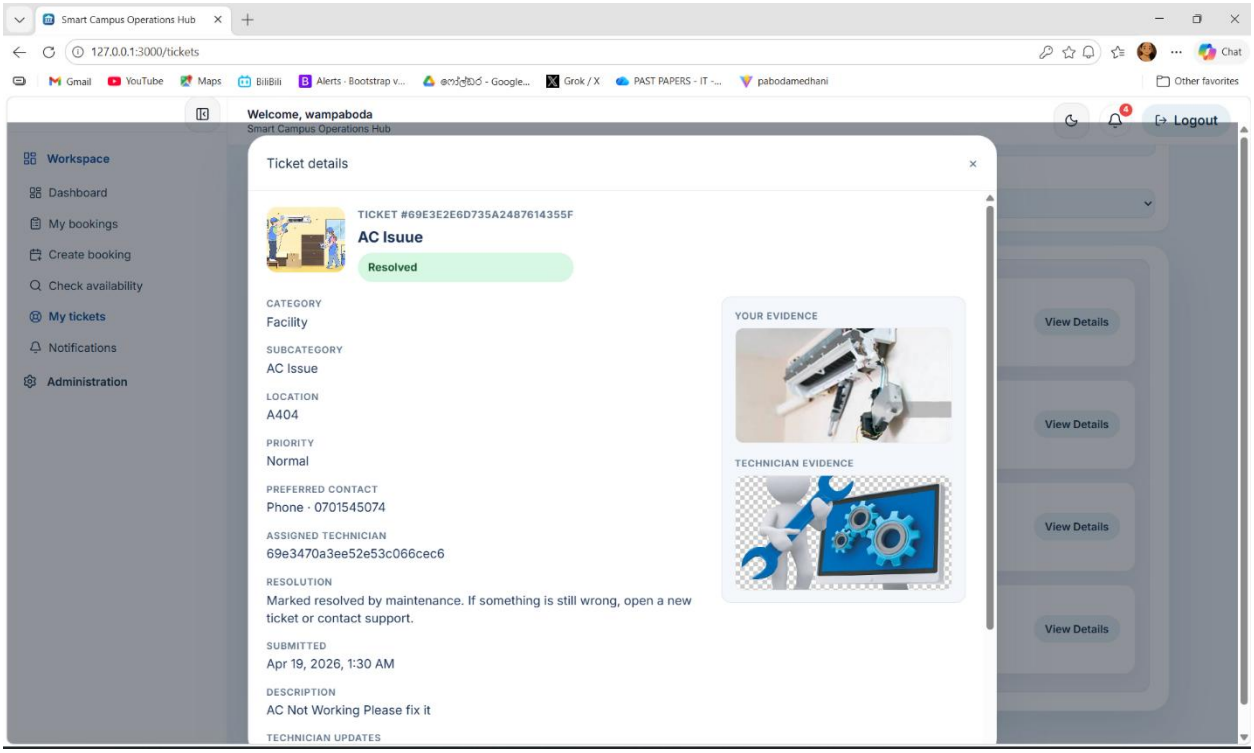
- User Portal – My tickets page



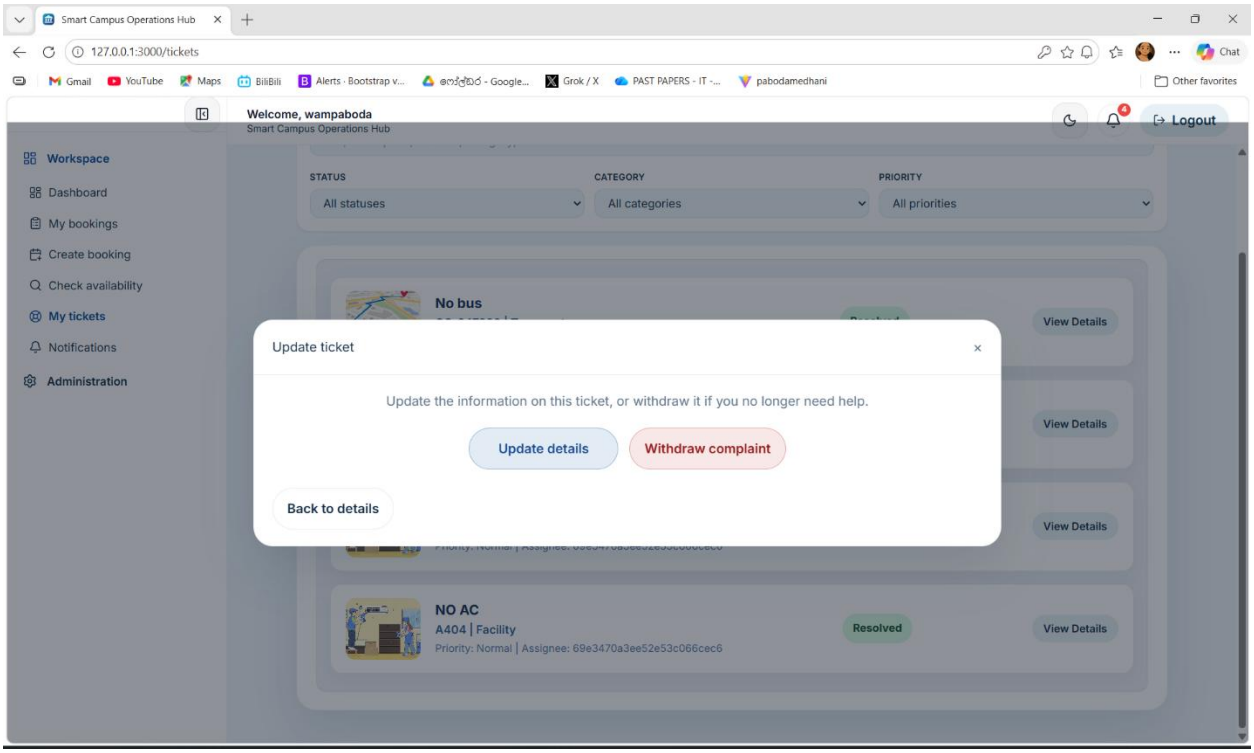
- Create Ticket Form

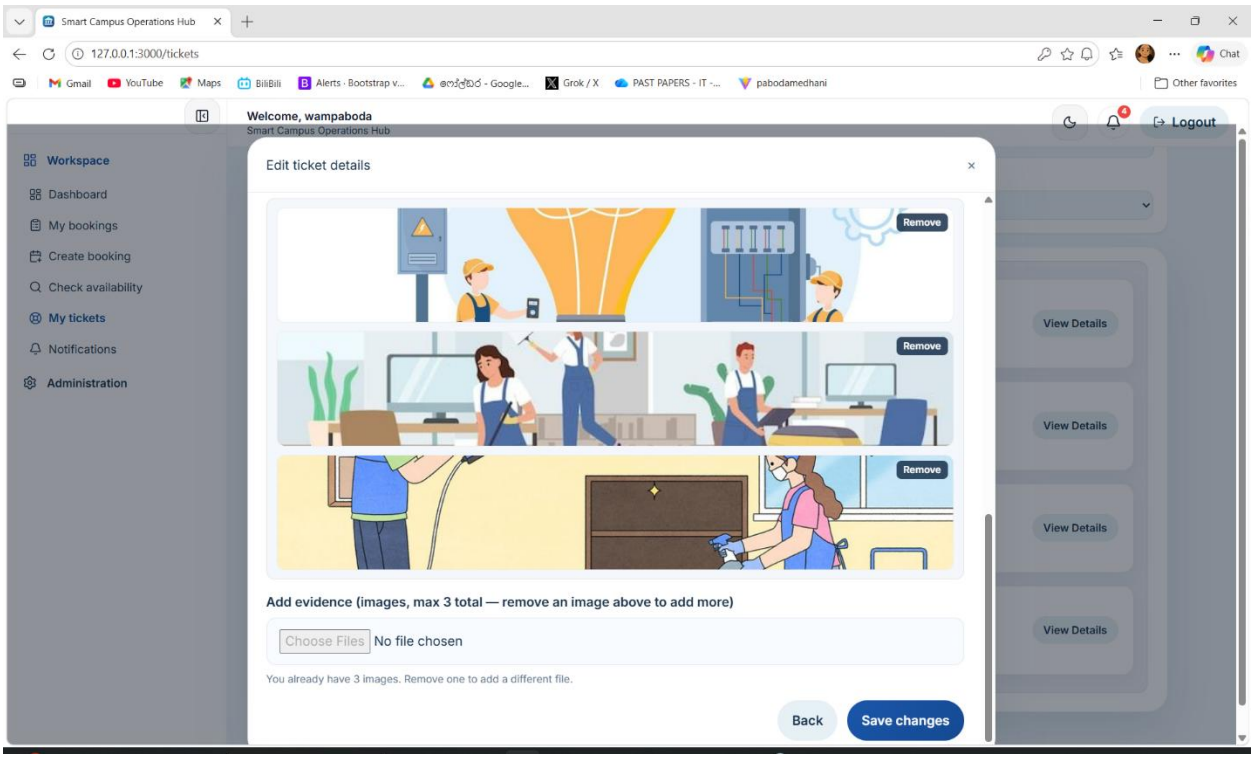


- Ticket Detail: Ticket Update

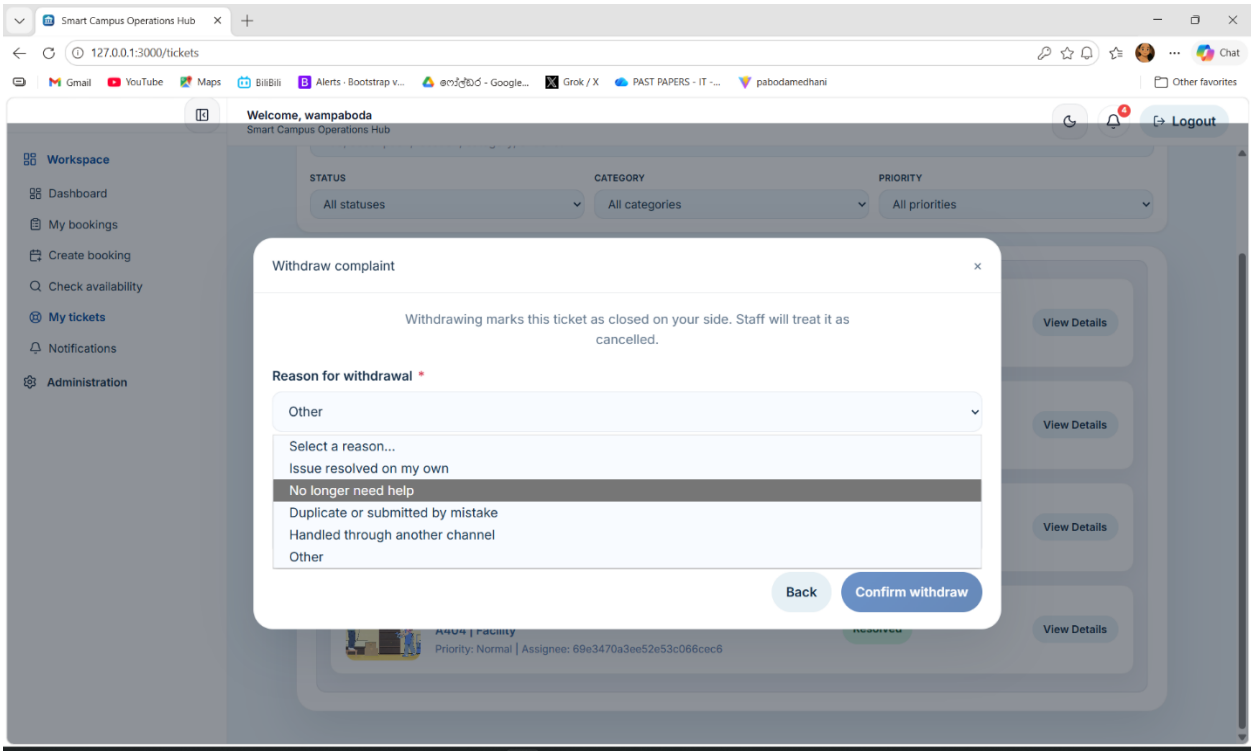


- Ticket Update

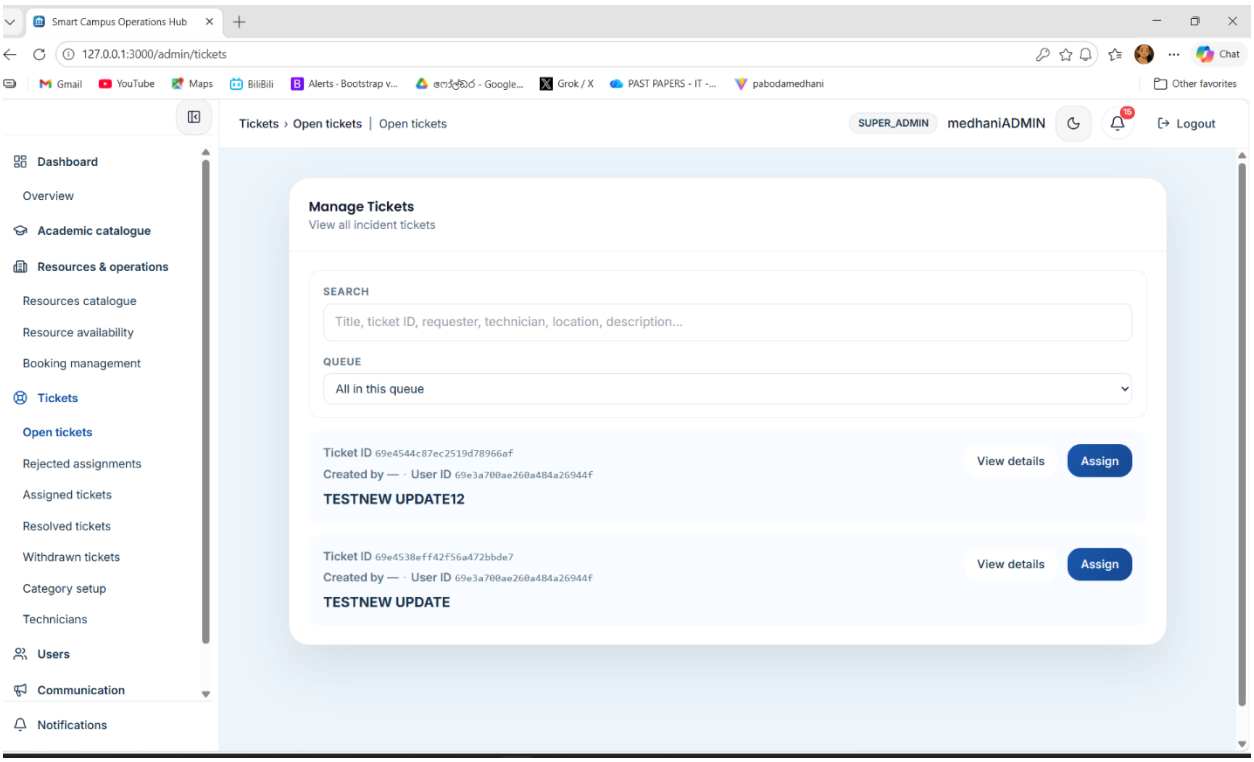




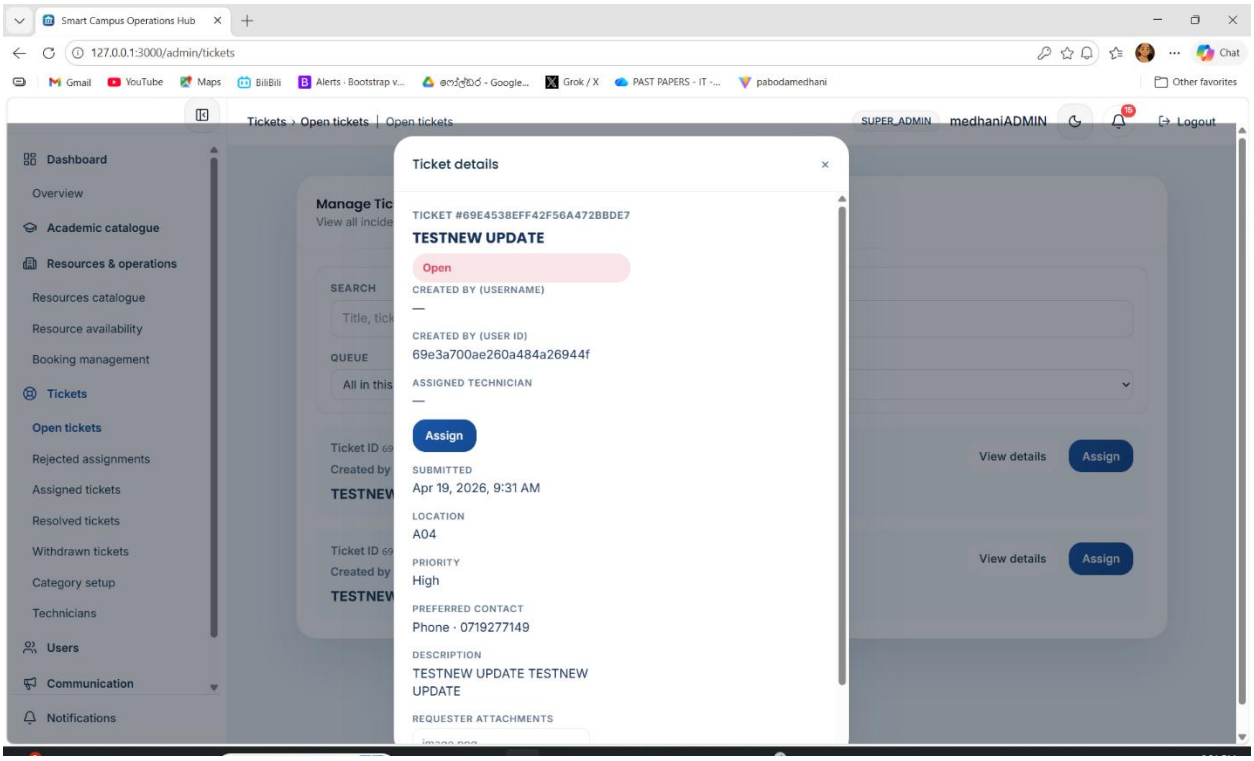
- Ticket withdrawals



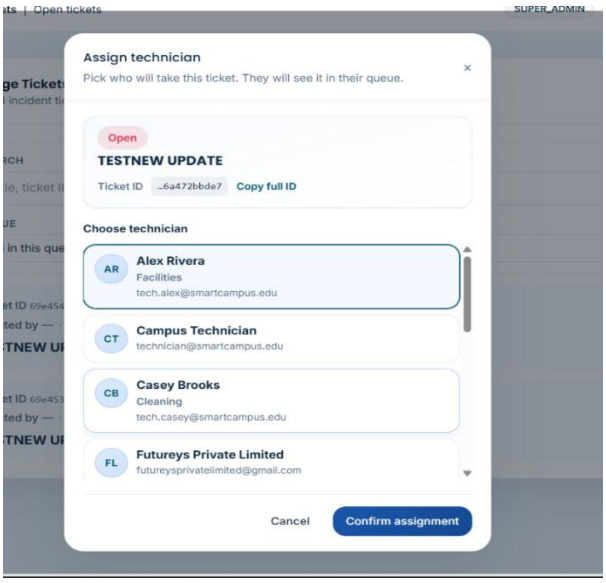
- Admin portal – Manage Tickets



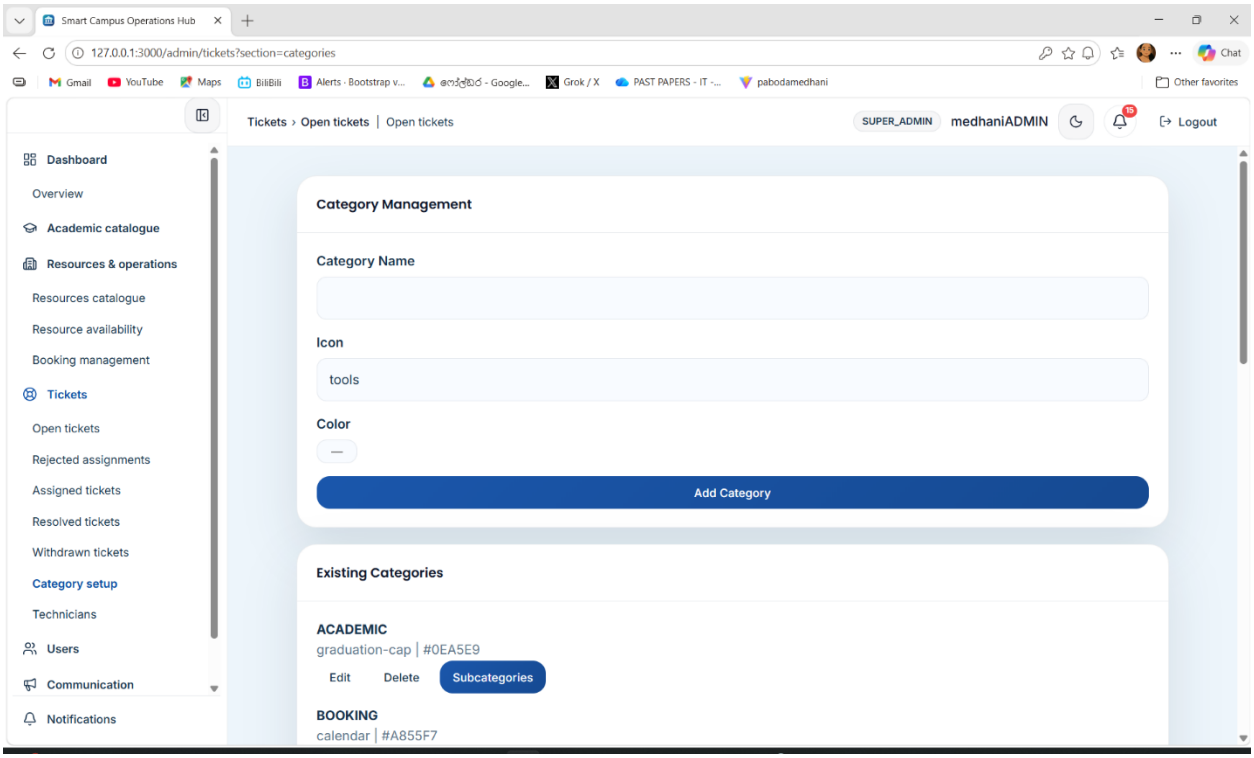
- Admin Portal – Ticket details



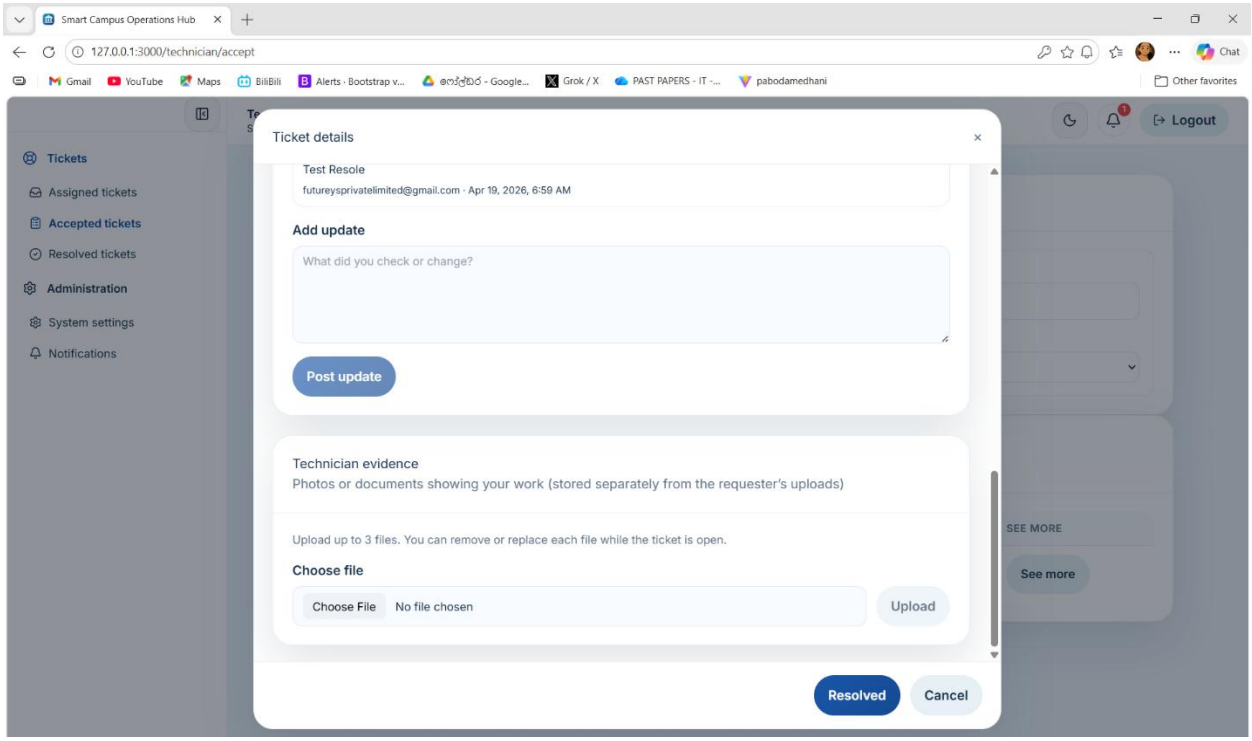
- Admin Portal – Technician Assign Form

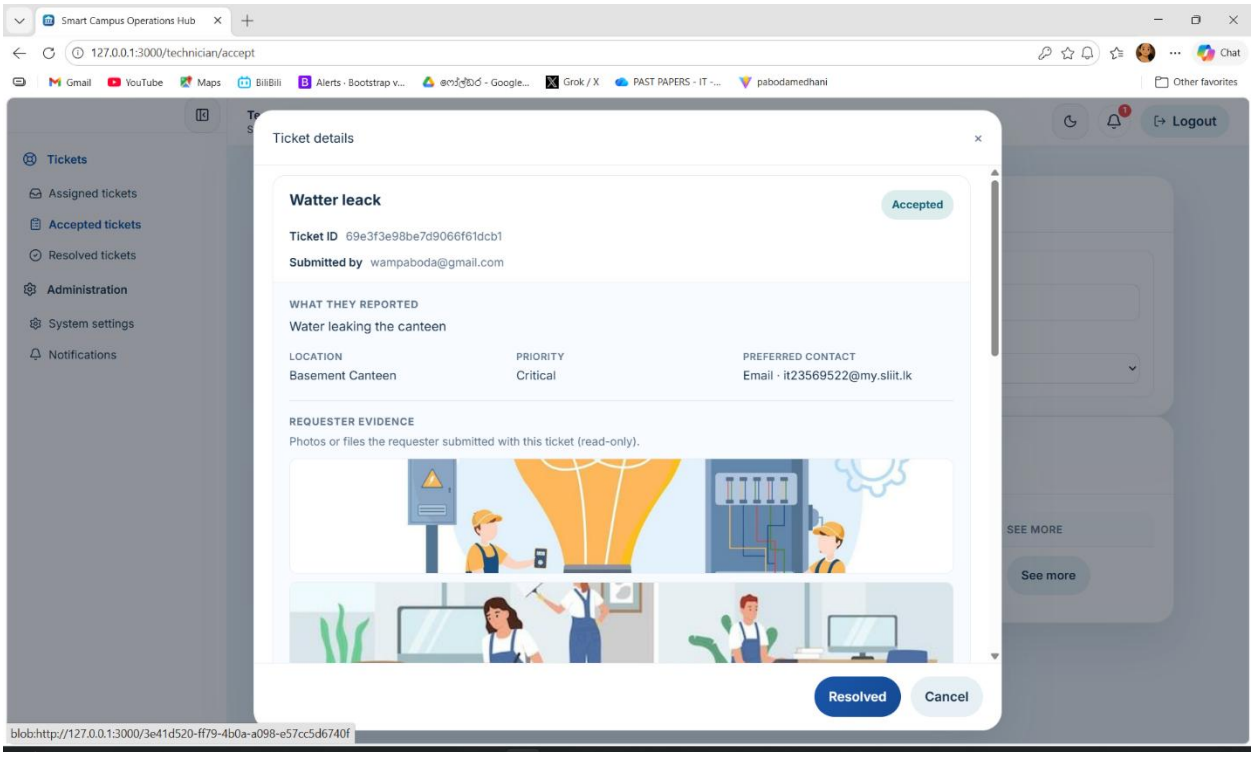


- Admin Portal – Category adding Form



- Technician Portal




Smart Campus Operations Hub
127.0.0.1:3000/technician/accept
Tickets
Assigned tickets
Accepted tickets
Resolved tickets
Administration
System settings
Notifications
Logout
Ticket details
Watter leack
Accepted
Ticket ID 69e3f3e98be7d9066f61dcb1
Submitted by wampaboda@gmail.com
WHAT THEY REPORTED
Water leaking the canteen
LOCATION
Basement Canteen
PRIORITY
Critical
PREFERRED CONTACT
Email · it23569522@my.sliit.lk
REQUESTER EVIDENCE
Photos or files the requester submitted with this ticket (read-only).
Resolved
Cancel
SEE MORE
See more
blob:http://127.0.0.1:3000/3e41d520-ff79-4b0a-a098-e57cc5d6740f

➢ Team Contribution Summary

Student ID	Core Feature	Backend Contribution	Frontend Contribution	Other
IT23580930	Authentication & Notification System	Implemented Google authentication support, notification APIs, unread count logic, and role-based notification delivery.	Developed Google Sign-In UI, notification bell, dropdown notifications, and connected frontend with backend APIs.	Improved user login experience, system communication, and dashboard usability.
IT23561670	Booking Management System	Implemented booking management logic to handle full booking lifecycle (create, view, approve/reject, cancel) and ensured validation for time-slot conflicts and resource availability	Developed CreateBookingPage.jsx, MyBookingsPage.jsx, and ApproveBookingsPage.jsx to support booking creation, user booking management, and admin approval workflow	Integrated booking workflow with resource availability and implemented API communication for real-time booking status updates
IT23585744	Facilities Catalogue, Resource Management, and Faculty Management	Implemented /api/v1/resources and /api/v1/faculties APIs, MongoDB-backed models, filtering, validation, availability checks, soft-delete handling, and academic cache refresh logic	Developed ManageResourcesPage.jsx, ResourceAvailabilityPage.jsx, and FacultiesAdminPage.jsx with CRUD flows, search and filtering, availability handling, and live API integration	Integrated resource data into booking workflows and faculty master data into academic and registration workflows

IT23569522	Incident & Ticketing System	Designed and implemented **REST API endpoints** under /api/v1/tickets Developed **CRUD operations** for ticket management Implemented **ticket lifecycle workflow/state machine** (Open → Assigned → Accepted → In Progress → Resolved / Withdrawn) Built **file upload APIs** for user and technician evidence Designed **MongoDB schema** for tickets, attachments, and status tracking Implemented **search and filtering logic** for tickets Developed **data normalization logic** for consistent frontend integration Contributed to **notification handling system** for ticket updates	Developed **MyTicketsPage.jsx** to handle full ticket lifecycle (create, view, update, withdraw) Implemented **form validation** using React Hook Form and Zod Integrated **AI-based category suggestion** (getTicketSuggestion) Built **image upload system** (max 3 images with preview & removal support) Implemented **PDF export feature** for resolved tickets Developed **Technician Dashboard** for task handling, notes, and evidence uploads Built **Admin ManageTicketsPage** for queue management, assignment, and audit tracking Created **API integration layer** (ticketService.js) Designed **responsive UI with role-based navigation**	Implemented **metadata parsing system** (location, priority, contact details) Added **status mapping for user-friendly UI labels** Improved **UI/UX with responsive design (Grid & Flexbox)** Ensured **end-to-end frontend-backend integration** Added **innovative features** (AI suggestions, PDF export)

7. GitHub

- GitHub Repo link :
  - https://github.com/LakviduUpasara/it3030-paf-2026-smart-malabe-group05
- OneDrive Link :
  - https://mysliit-my.sharepoint.com/my?id=%2Fpersonal%2Fit23580930%5Fmy%5Fsliit%5Flk%2FDocuments%2FY3S1%2EIT%2EWE%2E01%2E01%5FGrop%5F05&ga=1

- Image of the GitHub Commit Tree:

STUDENT ID	GITHUB USER NAME
IT23580930	LakviduUpasara
IT23561670	hiruniNwijesinghe
IT23585744	SupunKalharaJayasinghe
IT23569522	PabodaWA

```
GRAPH                                                                    Auto
update suggestion feature PabodaWA
ticket form PabodaWA
connect to mongodb PabodaWA
upadated backend PabodaWA
update PabodaWA
Update AdminDashboardPage.jsx PabodaWA
connect backend and frontend PabodaWA
backend connected to the front end PabodaWA
update ticket page PabodaWA
Ticket form PabodaWA
update PabodaWA
Update pom.xml PabodaWA
frontend PabodaWA
service package updated PabodaWA
Update TicketController.java PabodaWA
sevices done PabodaWA
dto update PabodaWA
Repository done PabodaWA
entity done PabodaWA
backend file creating PabodaWA
chore(frontend): improve root frontend scripts and refine landing hero Lakvidu Rathnayake
refactor(frontend): align auth pages with centered route-based flow Lakvidu Rathnayake
feat(frontend): add branded navbar identity and public nav icons Lakvidu Rathnayake
refactor(frontend): polish landing hero layout and branding copy Lakvidu Rathnayake
update navbar and move login to popup modal Lakvidu Rathnayake
Image change and full set background in Login UI Lakvidu Rathnayake
Set Sign Up button Lakvidu Rathnayake
Add google icons  and google login,apple signup ui Lakvidu Rathnayake
Create Login Dashbord and  Login Lakvidu Rathnayake
Setup Folder Stucter Lakvidu Rathnayake
```


```
GRAPH                                                                    Auto
update suggestion feature PabodaWA
ticket form PabodaWA
connect to mongodb PabodaWA
upadated backend PabodaWA
update PabodaWA
Update AdminDashboardPage.jsx PabodaWA
connect backend and frontend PabodaWA
backend connected to the front end PabodaWA
update ticket page PabodaWA
Ticket form PabodaWA
update PabodaWA
Update pom.xml PabodaWA
frontend PabodaWA
service package updated PabodaWA
Update TicketController.java PabodaWA
sevices done PabodaWA
dto update PabodaWA
Repository done PabodaWA
entity done PabodaWA
backend file creating PabodaWA
chore(frontend): improve root frontend scripts and refine landing hero Lakvidu Rathnayake
refactor(frontend): align auth pages with centered route-based flow Lakvidu Rathnayake
feat(frontend): add branded navbar identity and public nav icons Lakvidu Rathnayake
refactor(frontend): polish landing hero layout and branding copy Lakvidu Rathnayake
update navbar and move login to popup modal Lakvidu Rathnayake
Image change and full set background in Login UI Lakvidu Rathnayake
Set Sign Up button Lakvidu Rathnayake
Add google icons  and google login,apple signup ui Lakvidu Rathnayake
Create Login Dashbord and  Login Lakvidu Rathnayake
Setup Folder Stucter Lakvidu Rathnayake
```

8. References

[1] Mozilla Developer Network (MDN), “HTML, CSS, and JavaScript Documentation.” [Online]. Available: https://developer.mozilla.org

[2] React Documentation, “React – A JavaScript Library for Building User Interfaces.” [Online]. Available: https://react.dev

[3] Axios, “Axios HTTP Client Documentation.” [Online]. Available: https://axios-http.com

[4] Bootstrap, “Bootstrap CSS Framework Documentation.” [Online]. Available: https://getbootstrap.com

[5] Spring Boot Documentation, “Building REST APIs with Spring Boot.” [Online]. Available: https://spring.io/projects/spring-boot

[6] GitHub, “Git Version Control Platform.” [Online]. Available: https://github.com

[7] Firebase Documentation, “Firebase for Authentication and Hosting.” [Online]. Available: https://firebase.google.com/docs

[8] W3Schools, “Web Development Tutorials.” [Online]. Available: https://www.w3schools.com

[9] MongoDB Documentation, “MongoDB Official Documentation.” [Online]. Available: https://www.mongodb.com/docs

[10] Google Developers, “Google Identity Services Documentation.” [Online]. Available: https://developers.google.com/identity

[11] Playwright Documentation, “Playwright End-to-End Testing Framework.” [Online]. Available: https://playwright.dev

[12] Oracle, “Java Documentation.” [Online]. Available: https://docs.oracle.com/en/java/

[13] Spring Security Documentation, “Spring Security Reference.” [Online]. Available: https://spring.io/projects/spring-security

[14] Vite Documentation, “Vite Frontend Build Tool.” [Online]. Available: https://vitejs.dev

[15] npm, “Node Package Manager Documentation.” [Online]. Available: https://docs.npmjs.com